KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
http://www.polinema.ac.id

Nama/NIM : Claudya Destine Julia Handoko/2341760008
Mata Kuliah **:** Pemrograman Web Lanjut (PWL)
Program Studi **:** D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester **:** 4 (empat) / 6 (enam)
Pertemuan ke- **:** 10 (tujuh)

JOBSHEET 10

RESTFUL API

Sebelumnya kita sudah membahas mengenai *authentication*, *authorization*, dan *middleware* pada Laravel. Dimana kita telah membuat fungsi login, register, logout, serta pemilihan role dan penerapan session pada halaman web. Pada pertemuan kali ini, kita akan mempelajari penerapan RESTFUL API di dalam project Laravel.

Sebelum kita masuk materi, kita buat dulu project baru yang akan kita gunakan untuk membangun aplikasi sederhana dengan topik *Point of Sales (PoS)*, sesuai dengan `Studi Kasus PWL.pdf`. Jadi kita bikin project Laravel 10 dengan nama **PWL_POS.**

Project **PWL_POS** akan kita gunakan sampai pertemuan 12 nanti, sebagai project yang akan kita pelajarai

A. RESTFUL API

Representational State Transfer (REST) adalah gaya arsitektur perangkat lunak yang mendefinisikan seperangkat prinsip untuk merancang jaringan aplikasi terdistribusi. RESTful API adalah aplikasi pemrograman antarmuka yang mengikuti prinsip-prinsip REST untuk mentransfer data antara klien dan server.

RESTful API adalah salah satu arsitektur dalam API (*Application Program Interface*) yang menggunakan request HTTP untuk mengakses data. Data diakses dengan menggunakan HTTP method GET, PUT, POST dan DELETE yang merujuk pada operasi pembacaan, pembaruan, pembuatan dan penghapusan pada resource. Selain HTTP method, dalam RESTful atau REST digunakan juga HTTP response untuk mendefinisikan respon data yang dikembalikan. Format respon yang umum digunakan berupa JSON (Javascript Object Notation).

B. JSON Web Token (JWT)

JWT adalah singkatan dari JSON Web Token. Ini adalah standar terbuka (RFC 7519) yang mendefinisikan format token yang kompak dan mandiri untuk mentransfer klaim antara dua pihak. JWT sering digunakan dalam otentikasi dan pertukaran informasi yang aman di lingkungan yang tidak terpercaya, seperti internet.

JWT terdiri dari tiga bagian yang dipisahkan oleh titik ("."): header, payload, dan signature. Setiap bagian ini terdiri dari data JSON yang dienkripsi menggunakan algoritma tertentu dan kemudian disatukan untuk membentuk token yang lengkap. Header berisi jenis token dan tipe algoritma yang digunakan untuk enkripsi. Payload berisi klaim atau informasi yang ingin disampaikan. Signature digunakan untuk memverifikasi bahwa token belum berubah dan datanya berasal dari sumber yang dipercayai.

JWT sering digunakan dalam sistem otentikasi dan otorisasi modern, seperti aplikasi web dan layanan web API, karena fleksibilitasnya dalam menyampaikan informasi terenkripsi secara ringkas.

Kita dapat menggunakan JWT untuk:

- Authentication
  Ketika pengguna melakukan authentication dan mendapatkan token, maka setiap permintaan berikutnya akan menyertakan token tersebut, dan memungkinkan pengguna untuk mengakses route, service, dan resources yang diizinkan.
- Pertukaran informasi
  JSON Web Token adalah cara yang baik untuk mengirimkan informasi antar pihak dengan aman. Dengan token yang sudah ditandatangani dengan algoritma RSA, maka kita bisa tahu siapa yang melakukan request tersebut.

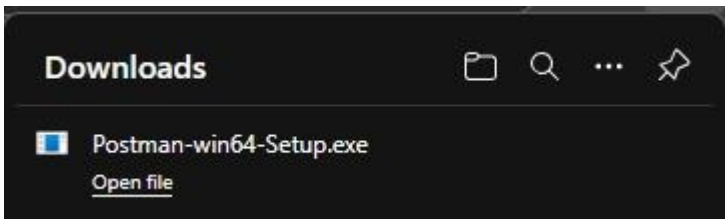
  Berikut adalah cara kerja JWT :

JWT (JSON Web Token) adalah cara untuk mentransfer informasi antara dua pihak secara aman sebagai objek JSON. Ini terdiri dari tiga bagian: header, payload, dan signature. Setelah pengguna berhasil autentikasi, server menghasilkan token JWT yang disematkan dalam permintaan HTTP. Server kemudian memvalidasi token untuk memberikan akses ke sumber daya yang diminta. Ini memberikan autentikasi yang aman dan stateless tanpa memerlukan penyimpanan status sesi di server.

Praktikum 1 – Membuat RESTful API Register

1. Sebelum memulai membuat REST API, terlebih dahulu download aplikasi Postman di https://www.postman.com/downloads.

   Aplikasi ini akan digunakan untuk mengerjakan semua tahap praktikum pada Jobsheet ini.



2. Lakukan instalasi JWT dengan mengetikkan perintah berikut: `composer require tymon/jwt-auth:2.1.1` Pastikan Anda terkoneksi dengan internet.

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> composer require tymon/jwt-auth:2.1.1
./composer.json has been updated
Running composer update tymon/jwt-auth
Loading composer repositories with package information
Updating dependencies
Lock file operations: 4 installs, 0 updates, 0 removals
  - Locking lcobucci/clock (2.3.0)
  - Locking lcobucci/jwt (4.0.4)
  - Locking stella-maris/clock (0.1.7)
  - Locking tymon/jwt-auth (2.1.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 6 installs, 0 updates, 0 removals
  - Downloading stella-maris/clock (0.1.7)
  - Downloading lcobucci/clock (2.3.0)
  - Downloading phpoffice/phpexcel (1.8.1)
  - Downloading maatwebsite/excel (v1.1.5)
  - Downloading lcobucci/jwt (4.0.4)
  - Downloading tymon/jwt-auth (2.1.1)
  - Installing stella-maris/clock (0.1.7): Extracting archive
  - Installing lcobucci/clock (2.3.0): Extracting archive
  - Installing phpoffice/phpexcel (1.8.1): Extracting archive
  - Installing maatwebsite/excel (v1.1.5): Extracting archive
  - Installing lcobucci/jwt (4.0.4): Extracting archive
  - Installing tymon/jwt-auth (2.1.1): Extracting archive
```

```
  INFO  Discovering packages.

  barryvdh/laravel-dompdf ..... DONE
  jeroennoten/laravel-adminlte ..... DONE
  laravel/sail ..... DONE
  laravel/sanctum ..... DONE
  laravel/tinker ..... DONE
  nesbot/carbon ..... DONE
  nunomaduro/collision ..... DONE
  nunomaduro/termwind ..... DONE
  spatie/laravel-ignition ..... DONE
  tymon/jwt-auth ..... DONE
  yajra/laravel-datatables-buttons ..... DONE
  yajra/laravel-datatables-editor ..... DONE
  yajra/laravel-datatables-fractal ..... DONE
  yajra/laravel-datatables-html ..... DONE
  yajra/laravel-datatables-oracle ..... DONE

93 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

  INFO  No publishable resources for tag [laravel-assets].

Found 1 security vulnerability advisory affecting 1 package.
Run "composer audit" for a full list of advisories.
```

3. Setelah berhasil menginstall JWT, lanjutkan dengan publish konfigurasi file dengan perintah berikut:

   ```
   php artisan vendor:publish --
   provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"
   ```

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan vendor:publish --provider="Tymo
n\JWTAuth\Providers\LaravelServiceProvider"

  INFO  Publishing assets.

  Copying file [C:\laragon\www\PWL2025\Week10\Jobsheet10\vendor\tymon\jwt-auth\config\co
nfig.php] to [C:\laragon\www\PWL2025\Week10\Jobsheet10\config\jwt.php]  DONE
```

4. Jika perintah di atas berhasil, maka kita akan mendapatkan 1 file baru yaitu config/jwt.php. Pada file ini dapat dilakukan konfigurasi jika memang diperlukan.
5. Setelah itu jalankan peintah berikut untuk membuat secret key JWT. `php artisan jwt:secret`

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan jwt:secret
jwt-auth secret [5nXWIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JTy6ah] set s
uccessfully.
```

   Jika berhasil, maka pada file .env akan ditambahkan sebuah baris berisi nilai key JWT_SECRET.

```
61  JWT_SECRET=5nXWIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JTy6ah
62
```

6. Selanjutnya lakukan konfigurasi guard API. Buka config/auth.php. Ubah bagian ‘guards’ menjadi seperti berikut.

```
'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],
    'api' => [
        'driver' => 'jwt',
        'provider' => 'users',
    ],
],
```

Hasil:

```
38      'guards' => [
39          'web' => [
40              'driver' => 'session',
41              'provider' => 'users',
42          ],
43          'api' => [
44              'driver' => 'jwt',
45              'provider' => 'users',
46          ],
47      ],
```

7. Kita akan menambahkan kode di model UserModel, ubah kode seperti berikut:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Tymon\JWTAuth\Contracts\JWTSubject;
use Illuminate\Foundation\Auth\User as Authenticatable;

class UserModel extends Authenticatable implements JWTSubject
{

    public function getJWTIdentifier(){
        return $this->getKey();
    }

    public function getJWTCustomClaims(){
        return [];
    }

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
```

Hasil :

app > Models > UserModel.php > PHP Intelephense > UserModel

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Tymon\JWTAuth\Contracts\JWTSubject; // <--- Tambahkan ini

21 references | 0 implementations
class UserModel extends Authenticatable implements JWTSubject // <--- Implement JWTSubject
{
    public function getJWTIdentifier(): mixed
    {
        return $this->getKey();
    }

    /**
     * Return a key value array, containing any custom claims to be added to the JWT.
     */
    0 references | 0 overrides
    public function getJWTCustomClaims(): array
    {
        return [];
    }
}
```

8. Berikutnya kita akan membuat controller untuk register dengan menjalankan peintah berikut.

   ```
   php artisan make:controller Api/RegisterController
   ```

   Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama RegisterController.

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/RegisterController

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\Register
Controller.php] created successfully.
```

9. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```php
<?php

namespace App\Http\Controllers\Api;

use App\Models\UserModel;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class RegisterController extends Controller
{
    public function __invoke(Request $request)
    {
```

```
14          //set validation
15          $validator = Validator::make($request->all(), [
16              'username' => 'required',
17              'nama' => 'required',
18              'password' => 'required|min:5|confirmed',
19              'level_id' => 'required'
20          ]);
21
22          //if validations fails
23          if($validator->fails()){
24              return response()->json($validator->errors(), 422);
25          }
26
27          //create user
28          $user = UserModel::create([
29              'username' => $request->username,
30              'nama' => $request->nama,
31              'password'  => bcrypt($request->password),
32              'level_id' => $request->level_id,
33          ]);
34
35          //return response JSON user is created
36          if($user){
37              return response()->json([
38                  'success' => true,
39                  'user' => $user,
40              ], 201);
41          }
42
43          //return JSON process insert failed
44          return response()->json([
45              'success' => false,
46          ], 409);
47      }
48  }
```

Hasil :

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\UserModel;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Validator;

class RegisterController extends Controller
{
    public function register(Request $request)
    {
        // Validasi input
        $validator = Validator::make($request->all(), [
            'username' => 'required|string|unique:m_user,username',
            'nama' => 'required|string|max:255',
            'password' => 'required|string|min:5',
        ]);

        if ($validator->fails()) {
            return response()->json($validator->errors(), 422);
        }

        // Simpan user baru
        $user = UserModel::create([
            'username' => $request->username,
            'nama' => $request->nama,
            'password' => Hash::make($request->password),
            'level_id' => 2, // Default level_id kalau mau (sesuaikan dengan kebutuhanmu)
        ]);

        return response()->json([
            'status' => true,
            'message' => 'Registrasi berhasil!',
            'data' => $user
        ], 201);
    }
}
```

10. Selanjutnya buka routes/api.php, ubah semua kode menjadi seperti berikut.

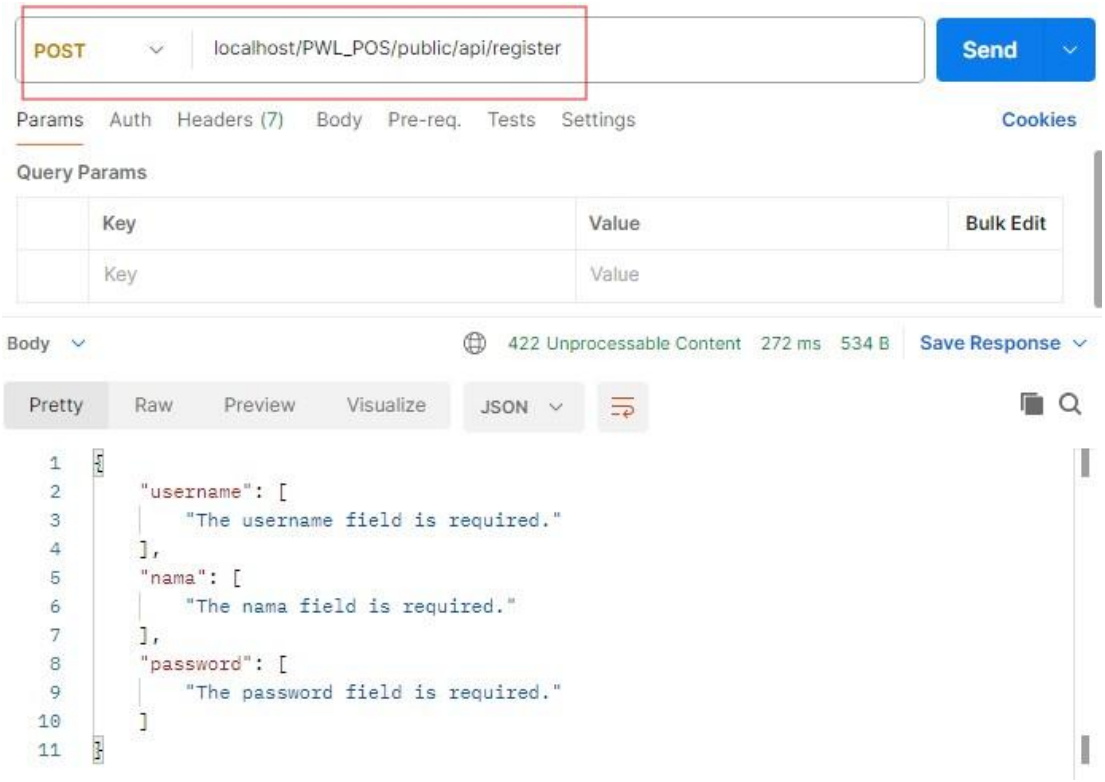
```php
<?php

use App\Http\Controllers\Api\RegisterController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

/*
|--------------------------------------------------------------------------
| API Routes
|--------------------------------------------------------------------------
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
```

Hasil :

routes > api.php

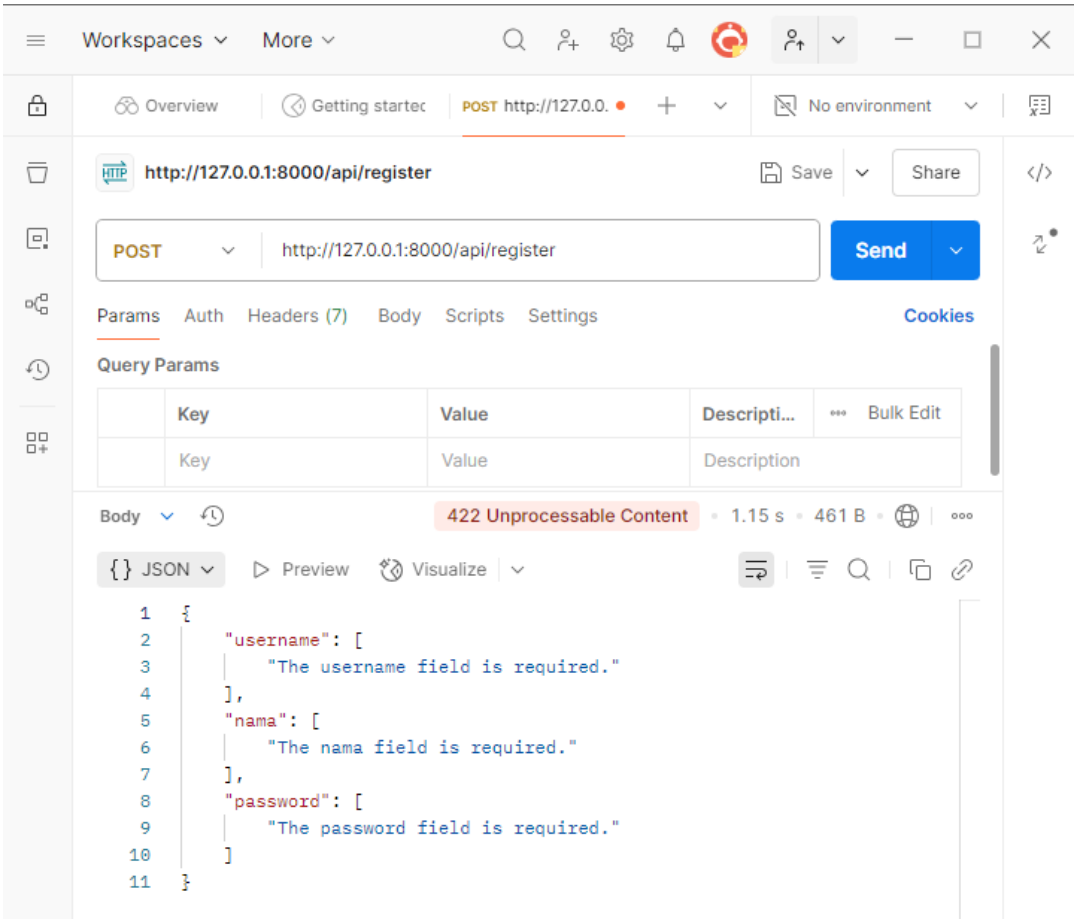
```php
<?php

use Iluminate\Http\Controllers\Api\RegisterController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;


/*
|--------------------------------------------------------------------------
| API Routes
|--------------------------------------------------------------------------
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)
->name('register');

```

11. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/register serta method POST. Klik Send.

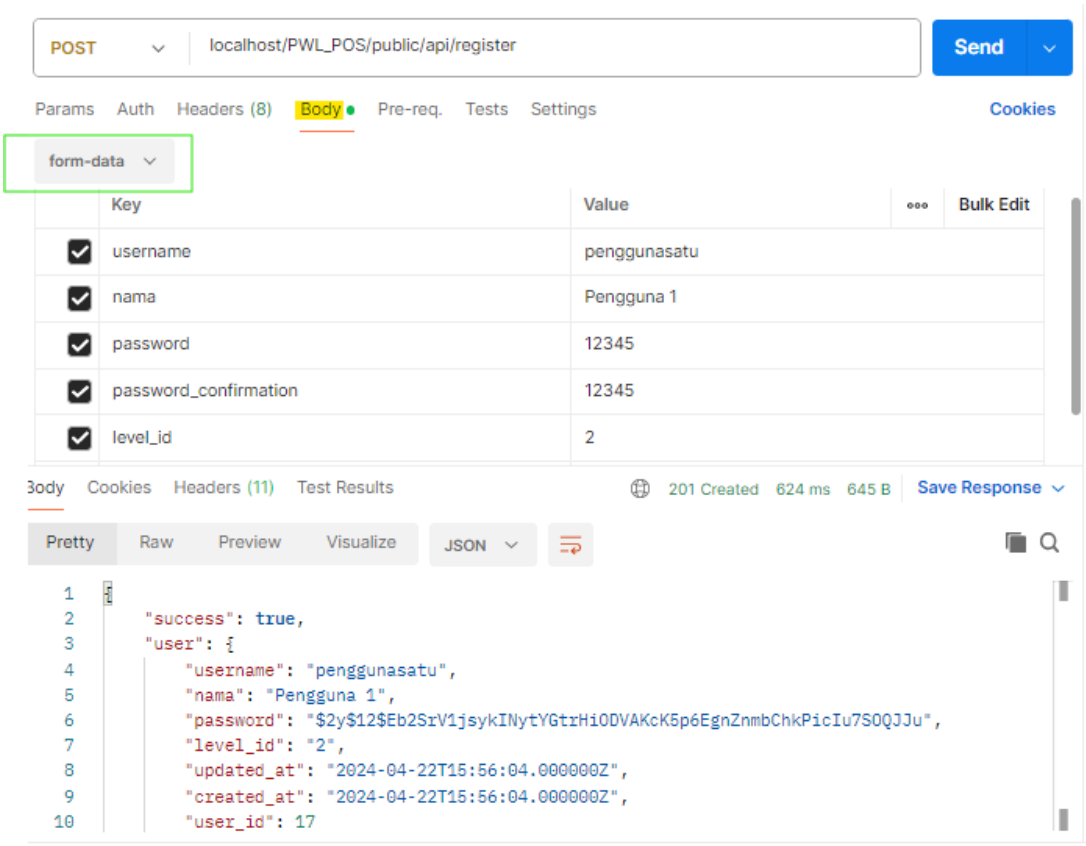


Jika berhasil akan muncul error validasi seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.

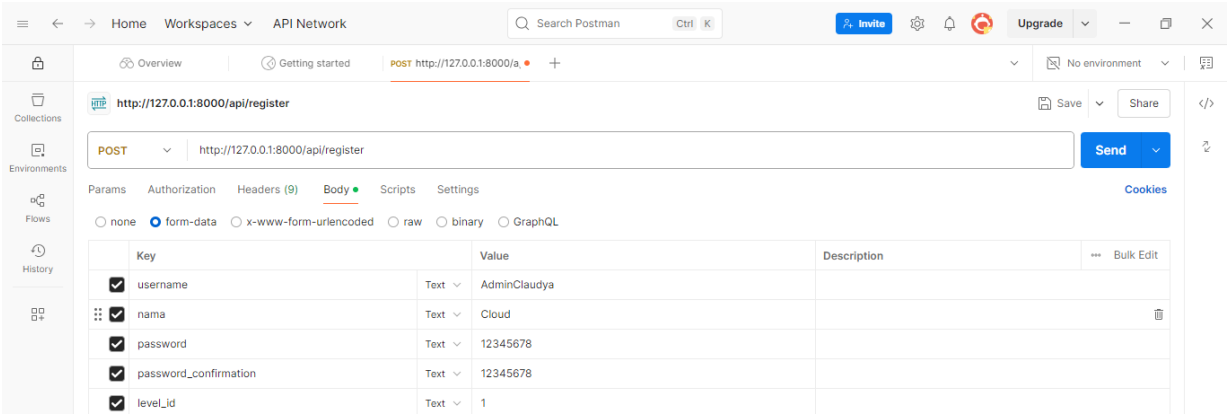


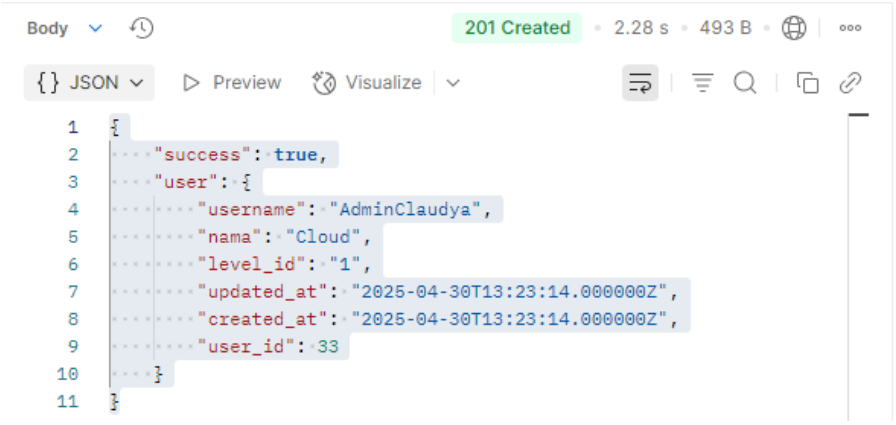
12. Sekarang kita coba masukkan data. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data registrasi menggunakan nilai yang Anda inginkan.

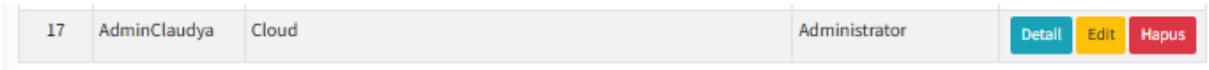


Setelah klik tombol Send, jika berhasil maka akan keluar pesan sukses seperti gambar di atas.

Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.







13. Lakukan commit perubahan file pada Github.

Praktikum 2 – Membuat RESTful API Login

1. Kita buat file controller dengan nama LoginController. `php artisan make:controller Api/LoginController`

   Jika berhasil maka akan ada tambahan controller pada folder Api dengan nama LoginController.

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/LoginController

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\LoginControlle
r.php] created successfully.
```

2. Buka file tersebut, dan ubah kode menjadi seperti berikut.

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

```

```php
class LoginController extends Controller
{
    public function __invoke(Request $request)
    {
        //set validation
        $validator = Validator::make($request->all(), [
            'username'     => 'required',
            'password'  => 'required'
        ]);

        //if validation fails
        if ($validator->fails()) {
            return response()->json($validator->errors(), 422);
        }

        //get credentials from request
        $credentials = $request->only('username', 'password');

        //if auth failed
        if(!$token = auth()->guard('api')->attempt($credentials)) {
            return response()->json([
                'success' => false,
                'message' => 'Username atau Password Anda salah'
            ], 401);
        }

        //if auth success
        return response()->json([
            'success' => true,
            'user'    => auth()->guard('api')->user(),
            'token'   => $token
        ], 200);
    }
}
```

Hasil :

app > Http > Controllers > Api > LoginController.php > PHP > LoginController > login()

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

0 references | 0 implementations
class LoginController extends Controller
{
    0 references | 0 overrides
```

```php
app > Http > Controllers > Api > LoginController.php > PHP Intelephense > LoginController > __invoke
2 references | 0 implementations
class LoginController extends Controller
{
    0 references | 0 overrides
    public function __invoke(Request $request): JsonResponse|mixed
    {
        // Set validation
        $validator = Validator::make($request->all(), [
            'username' => 'required',
            'password' => 'required',
        ]);

        // If validation fails
        if ($validator->fails()) {
            return response()->json($validator->errors(), 422);
        }

        // Get credentials from request
        $credentials = $request->only('username', 'password');

        // If auth failed
        if (!$token = auth()->guard('api')->attempt(credentials: $credentials)) {
            return response()->json([
                'success' => false,
                'message' => 'Username atau Password Anda salah'
            ], 401);
        }

        // If auth success
        return response()->json([
            'success' => true,
            'user' => auth()->guard('api')->user(),
            'token' => $token
        ], 200);
    }
}
```

3. Berikutnya tambahkan route baru pada file api.php yaitu /login dan /user.

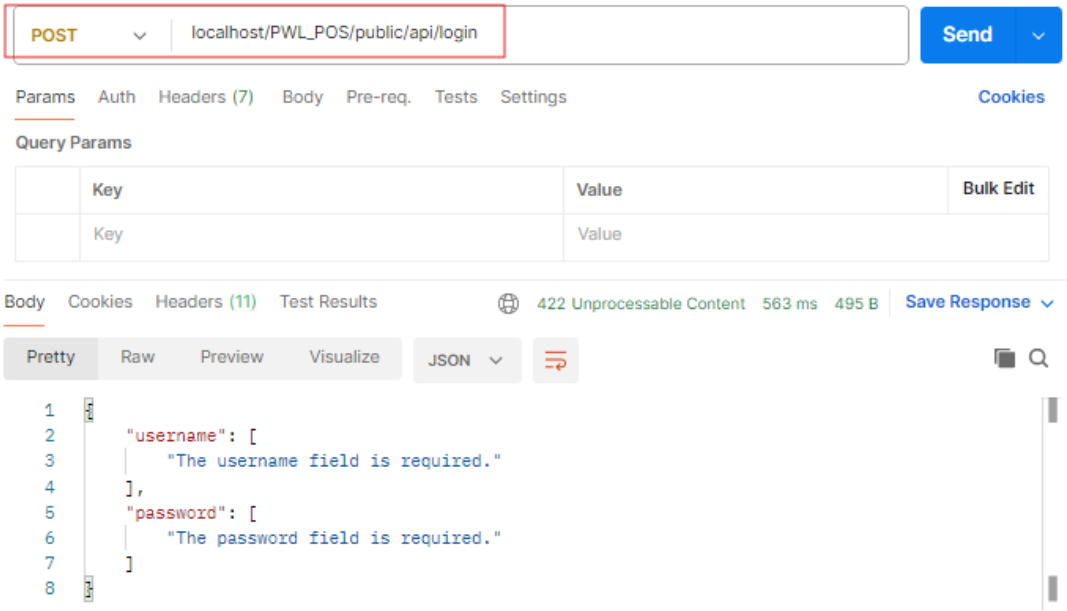
```php
use App\Http\Controllers\Api\LoginController;

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
Route::post('/login', App\Http\Controllers\Api\LoginController::class)->name('login');
Route::middleware('auth:api')->get('/user', function (Request $request) {
    return $request->user();
});
```

Hasil :

routes > api.php > ...

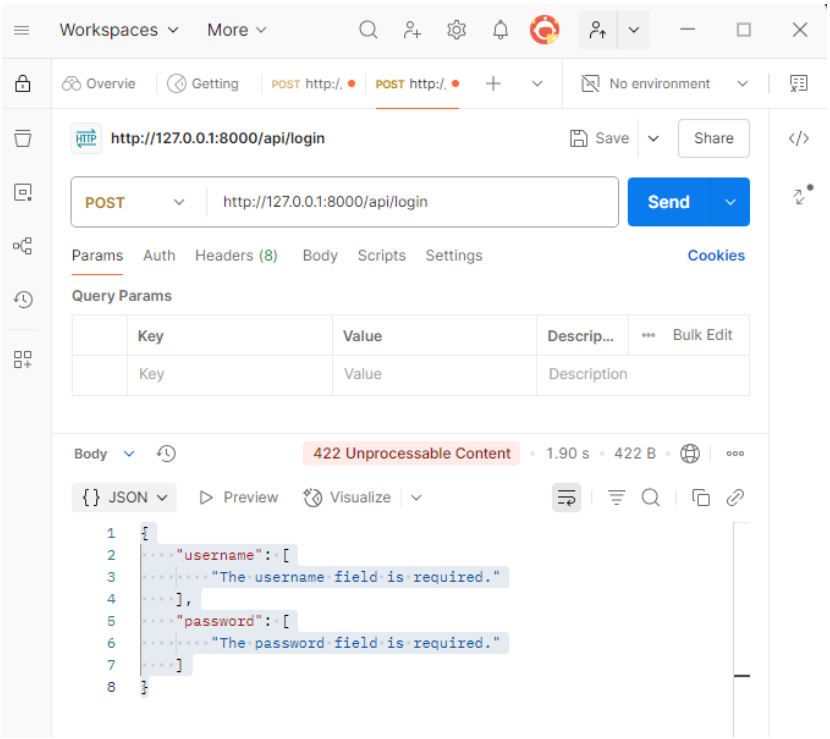
```php
<?php

use App\Http\Controllers\Api\RegisterController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\LoginController;


/*
|--------------------------------------------------------------------------
| API Routes
|--------------------------------------------------------------------------
|
| Here is where you can register API routes for your application. These
| routes are loaded by the RouteServiceProvider and all of them will
| be assigned to the "api" middleware group. Make something great!
|
*/

Route::post('/register', App\Http\Controllers\Api\RegisterController::class)->name('register');
Route::post('/login', App\Http\Controllers\Api\LoginController::class)->name('login');
Route::middleware('auth:api')->get('/user', function (Request $request): mixed {
    return $request->user();
});
```

4. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/login serta method POST. Klik Send.

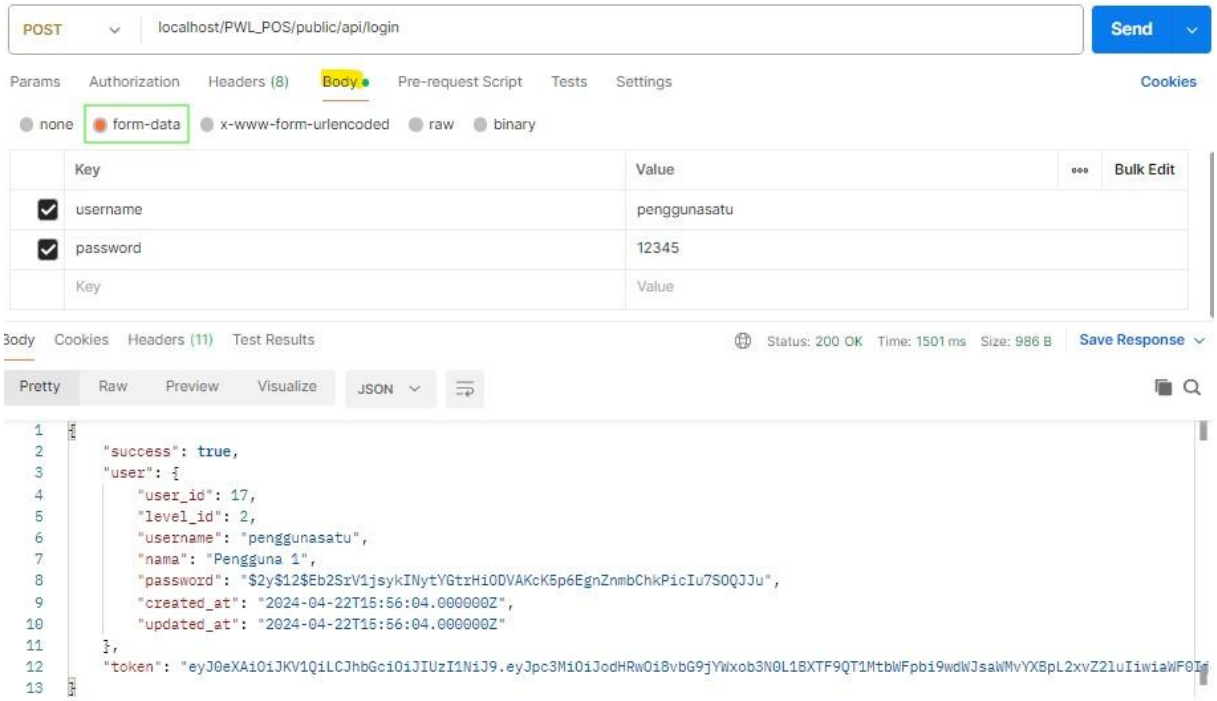


Jika berhasil akan muncul error validasi seperti gambar di atas.

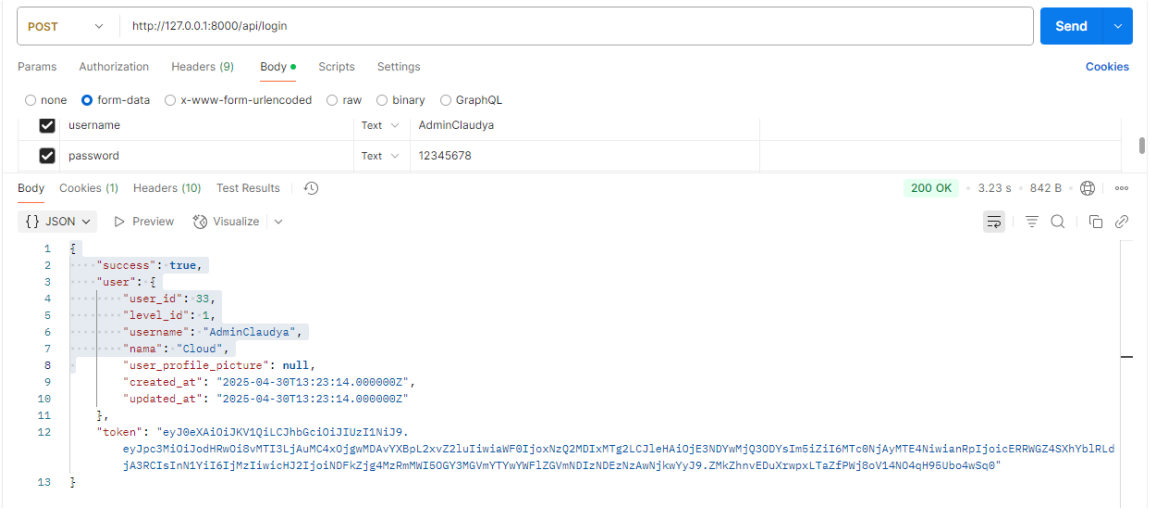
Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.



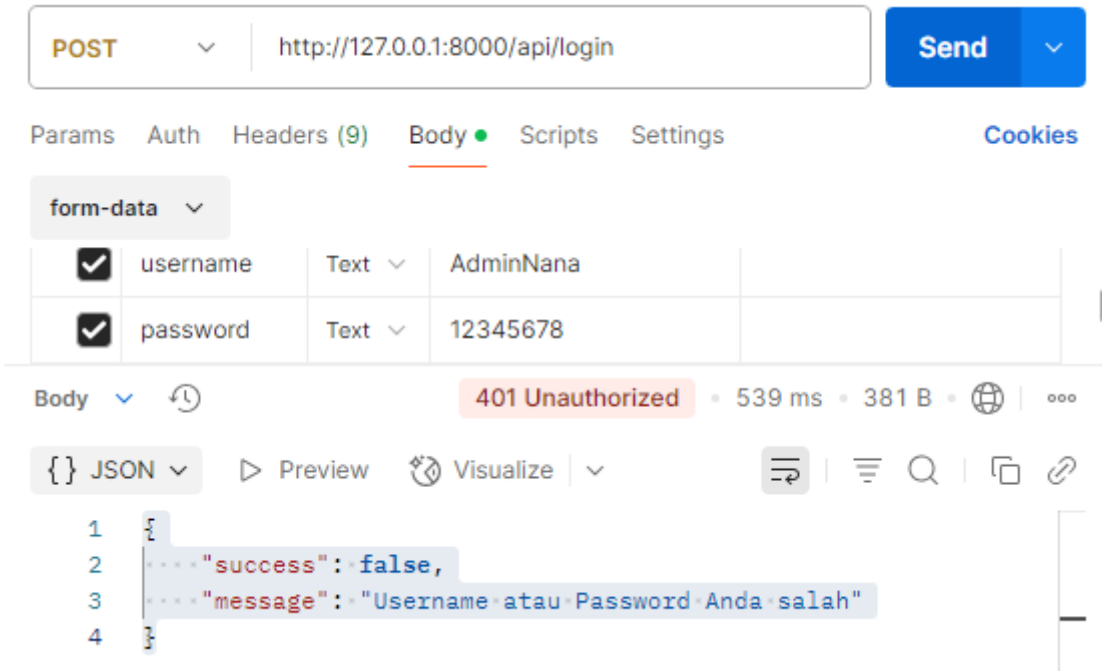
5. Selanjutnya, isikan username dan password sesuai dengan data user yang ada pada database. Klik tab Body dan pilih form-data. Isikan key sesuai dengan kolom data, serta isikan data user. Klik tombol Send, jika berhasil maka akan keluar tampilan seperti berikut. Copy nilai token yang diperoleh pada saat login karena akan diperlukan pada saat logout.


POST
localhost/PWL_POS/public/api/login
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
Key
Value
Bulk Edit
username
penggunasatu
password
12345
Key
Value
Body
Cookies
Headers (11)
Test Results
Status: 200 OK
Time: 1501 ms
Size: 986 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"success": true,
"user": {
"user_id": 17,
"level_id": 2,
"username": "penggunasatu",
"nama": "Pengguna 1",
"password": "$2y$12$Eb2SrV1jsykINytYGtrHiODVAKcK5p6EgnZnmbChkPicIu7SOQJJu",
"created_at": "2024-04-22T15:56:04.000000Z",
"updated_at": "2024-04-22T15:56:04.000000Z"
},
"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOi8vbG9jYWxob3N0L1BXTF9QT1MtbWFpbi9wdWJsaWMvYXBpL2xvZ2luIiwiaWF0I
}

Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.

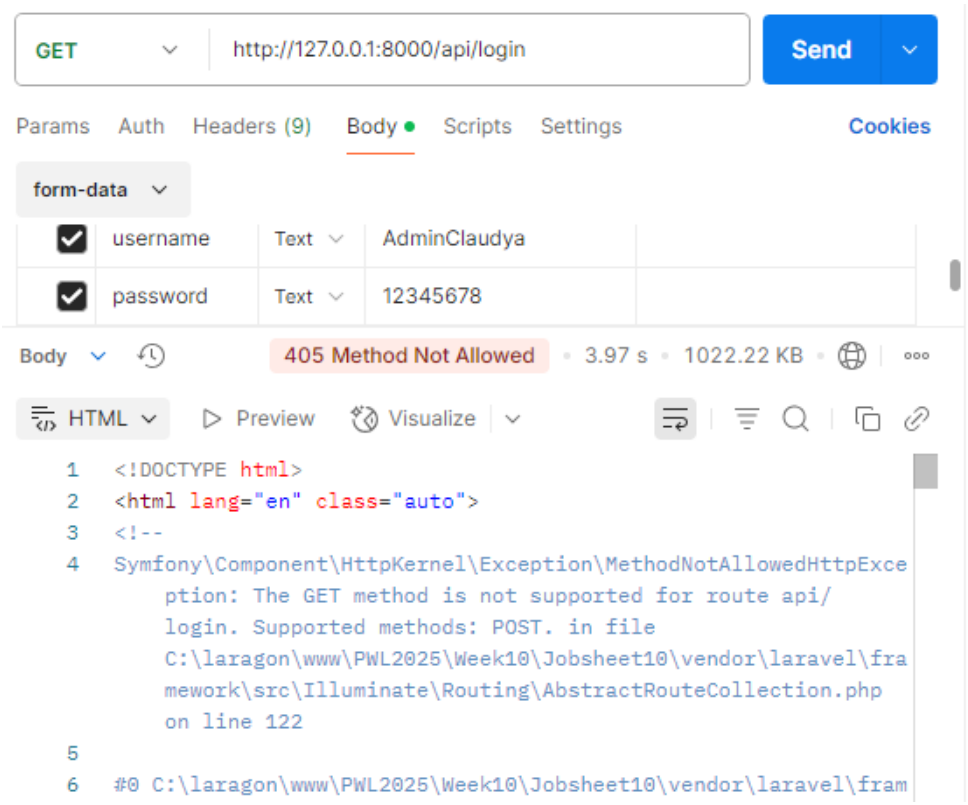


6. Lakukan percobaan yang untuk data yang salah dan berikan screenshoot hasil percobaan Anda.



7. Coba kembali melakukan login dengan data yang benar. Sekarang mari kita coba menampilkan data user yang sedang login menggunakan URL localhost/PWL_POS/public/api/user dan method GET. Jelaskan hasil dari percobaan tersebut.

Jawab : API login tidak mendukung metode GET. Rute `api/login` telah dikonfigurasi untuk hanya menerima permintaan POST, sehingga penggunaan GET akan menghasilkan error 405 "Method Not Allowed".



8. Lakukan commit perubahan file pada Github.

Praktikum 3 – Membuat RESTful API Logout

1. Tambahkan kode berikut pada file .env
   `JWT_SHOW_BLACKLIST_EXCEPTION=true`

```
61  JWT_SECRET=5nXWIhNTEwy6puOtFAnCK0YZXPERLi13cIG8GdM1rTLmk6azG9F0yPsEy6JTy6ah
62  JWT_SHOW_BLACKLIST_EXCEPTION=true
```

2. Buat Controller baru dengan nama LogoutController. `php artisan make:controller Api/LogoutController`

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/Logout
Controller

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\
Api\LogoutController.php] created successfully.
```

3. Buka file tersebut dan ubah kode menjadi seperti berikut.

```php
<?php

namespace App\Http\Controllers\Api;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use Tymon\JWTAuth\Facades\JWTAuth;
use Tymon\JWTAuth\Exceptions\JWTException;
use Tymon\JWTAuth\Exceptions\TokenExpiredException;
use Tymon\JWTAuth\Exceptions\TokenInvalidException;

class LogoutController extends Controller
{
    public function __invoke(Request $request)
    {
        //remove token
        $removeToken = JWTAuth::invalidate(JWTAuth::getToken());

        if($removeToken) {
            //return response JSON
            return response()->json([
                'success' => true,
                'message' => 'Logout Berhasil!',
            ]);
        }
    }
}
```

Hasil :

app > Http > Controllers > Api > LogoutController.php > PHP > LogoutController

```php
<?php

namespace App\Http\Controllers\Api;

use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use Tymon\JWTAuth\Facades\JWTAuth;
use Tymon\JWTAuth\Exceptions\JWTException;
use Tymon\JWTAuth\Exceptions\TokenExpiredException;
use Tymon\JWTAuth\Exceptions\TokenInvalidException;

1 reference | 0 implementations
class LogoutController extends Controller
{
    0 references | 0 overrides
    public function __invoke(Request $request): JsonResponse|mixed
    {
        //remove token
        $removeToken = JWTAuth::invalidate(JWTAuth::getToken());

        if($removeToken) {
            //return response JSON
            return response()->json([
                'success' => true,
                'message' => 'Logout Berhasil!',
            ]);
        }
    }
}
```

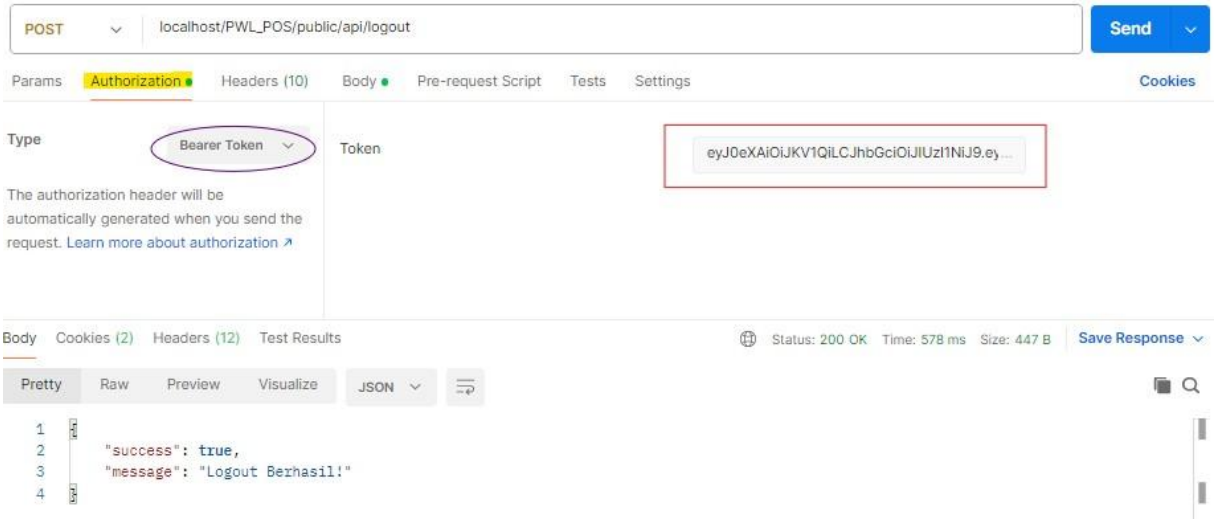
4. Lalu kita tambahkan routes pada api.php

```php
Route::post('/logout', App\Http\Controllers\Api\LogoutController::class)->name('logout');
```

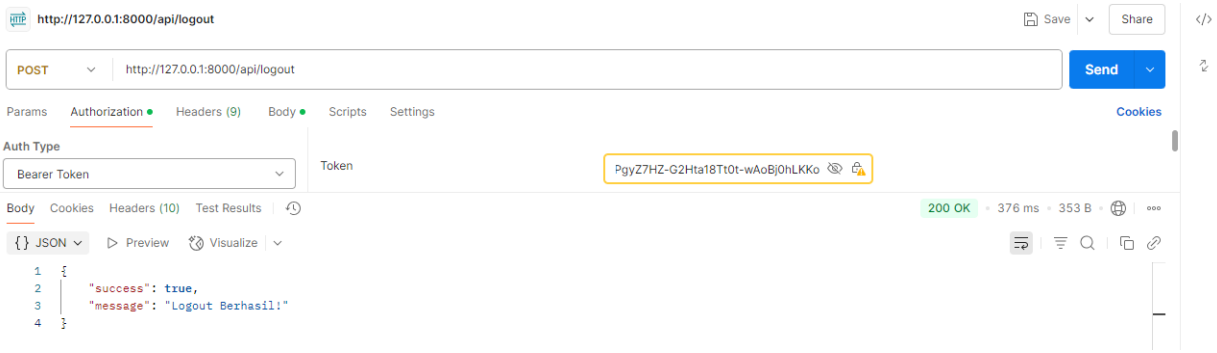
Hasil :

```php
Route::post('/logout', App\Http\Controllers\Api\LogoutController::class)->name('logout');
```

5. Jika sudah, kita akan melakukan uji coba REST API melalui aplikasi Postman. Buka aplikasi Postman, isi URL localhost/PWL_POS/public/api/logout serta method POST.
6. Isi token pada tab Authorization, pilih Type yaitu Bearer Token. Isikan token yang didapat saat login. Jika sudah klik Send.



Lakukan percobaan yang sama dan berikan screenshoot hasil percobaan Anda.



7. Lakukan commit perubahan file pada Github.

Praktikum 4 – Implementasi CRUD dalam RESTful API

Pada praktikum ini kita akan menggunakan tabel m_level untuk dimodifikasi menggunakan RESTful API.

1. Pertama, buat controller untuk mengolah API pada data level.

```
php artisan make:controller Api/LevelController
```

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/LevelContro
ller

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\L
evelController.php] created successfully.
```

2. Setelah berhasil, buka file tersebut dan tuliskan kode seperti berikut yang berisi fungsi CRUDnya.

```php
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\LevelModel;

class LevelController extends Controller
{
    public function index()
    {
        return LevelModel::all();
    }

    public function store(Request $request)
    {
        $level = LevelModel::create($request->all());
        return response()->json($level, 201);
    }

    public function show(LevelModel $level)
    {
        return LevelModel::find($level);
    }

    public function update(Request $request, LevelModel $level)
    {
        $level->update($request->all());
        return LevelModel::find($level);
    }

    public function destroy(LevelModel $user)
    {
        $user->delete();

        return response()->json([
            'success' => true,
            'message' => 'Data terhapus',
        ]);
    }
}
```

Hasil :

app > Http > Controllers > Api > LevelController.php > PHP > LevelController

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\LevelModel;

6 references | 0 implementations
class LevelController extends Controller
{
    public function index(): Collection
    {
        return LevelModel::all();
    }

    1 reference | 0 overrides
    public function store(Request $request): JsonResponse|mixed
    {
        $level = LevelModel::create($request->all());
        return response()->json($level, 201);
    }

    1 reference | 0 overrides
    public function show(LevelModel $level): array|Collection|LevelModel|Model|null
    {
        return LevelModel::find($level);
    }

    1 reference | 0 overrides
    public function update(Request $request, LevelModel $level): array|Collection|LevelModel|Model|null
    {
        $level->update($request->all());
        return LevelModel::find($level);
    }

    1 reference | 0 overrides
    public function destroy(LevelModel $user): JsonResponse|mixed
    {
        $user->delete();

        return response()->json([
            'success' => true,
            'message' => 'Data terhapus',
        ]);
    }
}
```

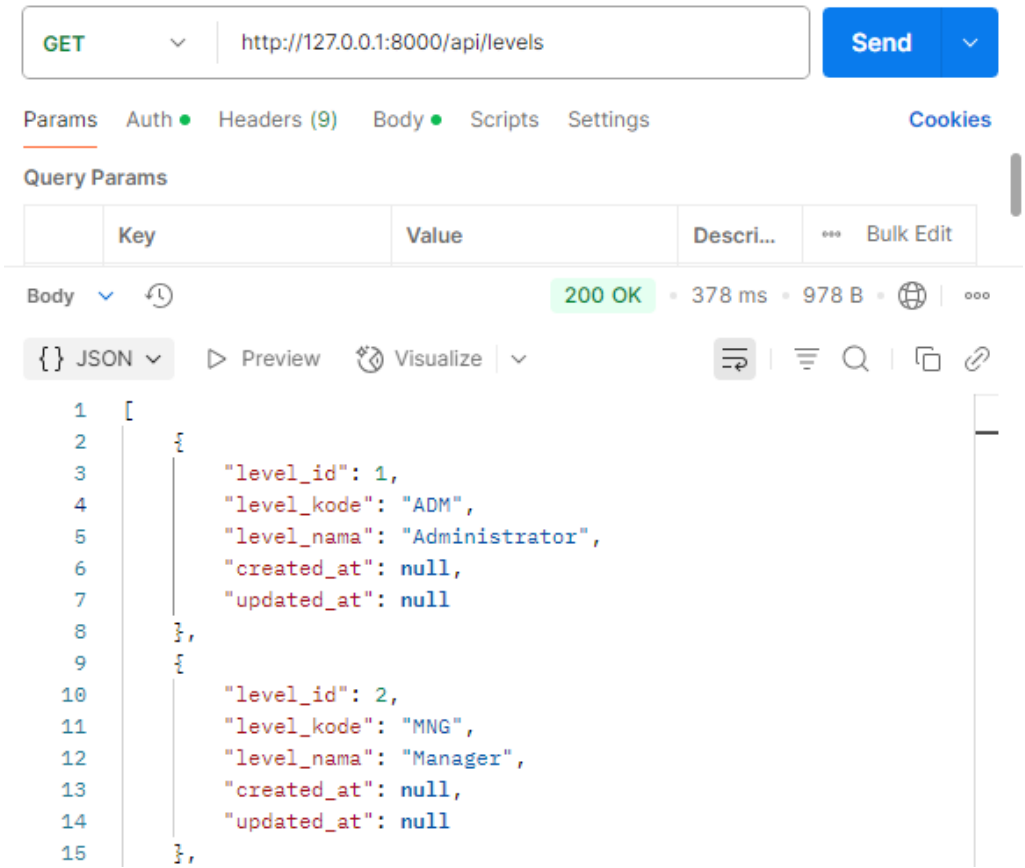
3. Kemudian kita lengkapi routes pada api.php.

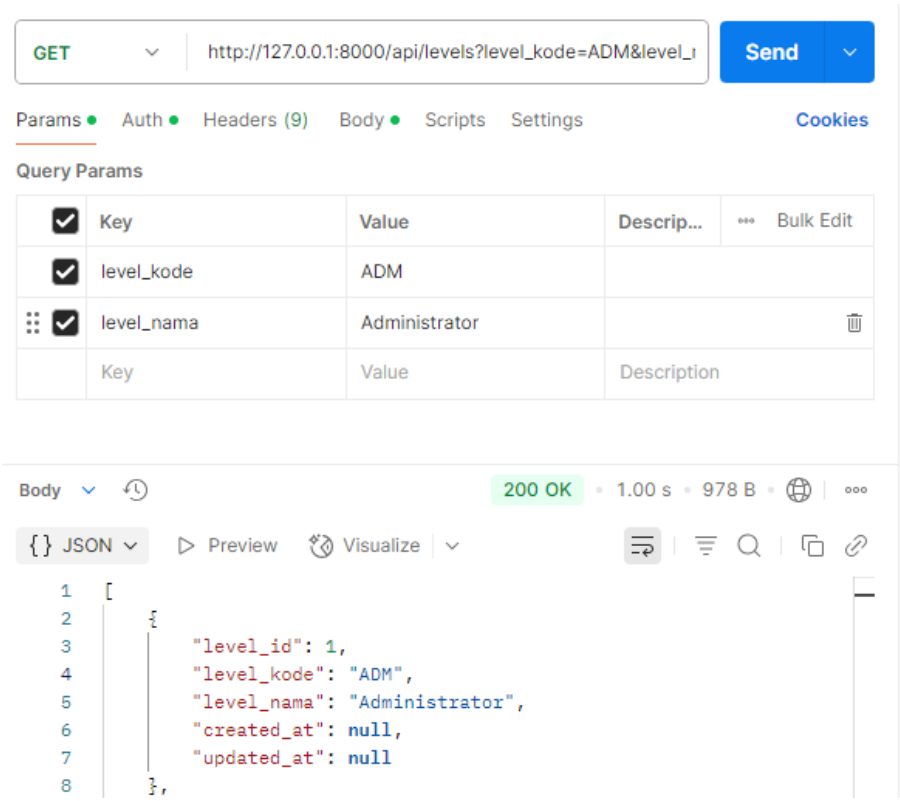
```php
use App\Http\Controllers\Api\LevelController;

Route::get('levels', [LevelController::class, 'index']);
Route::post('levels', [LevelController::class, 'store']);
Route::get('levels/{level}', [LevelController::class, 'show']);
Route::put('levels/{level}', [LevelController::class, 'update']);
Route::delete('levels/{level}', [LevelController::class, 'destroy']);
```

Hasil :

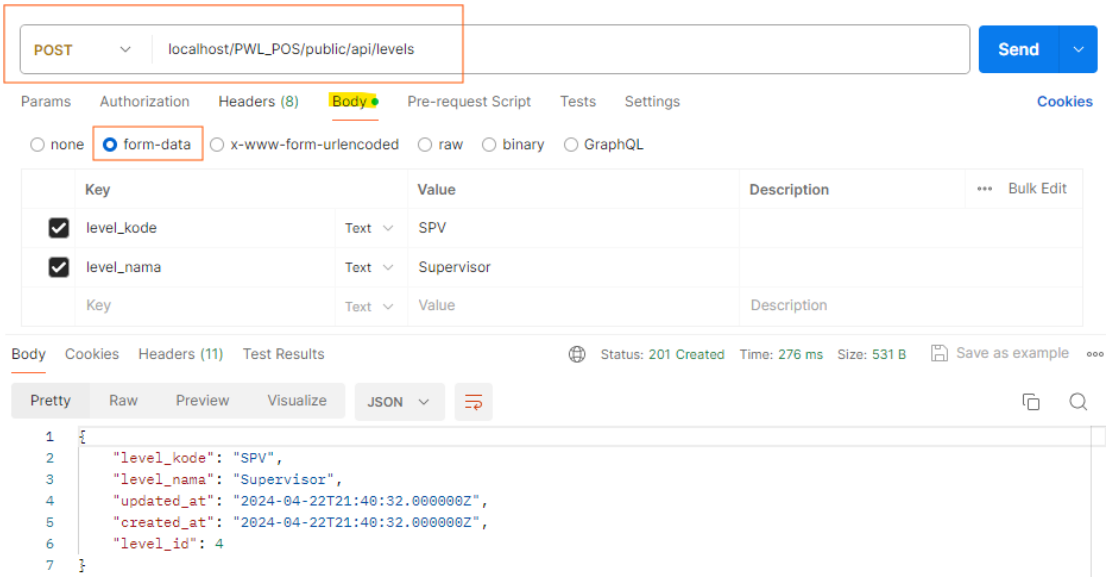
```
29  Route::get('levels', [LevelController::class, 'index']);
30  Route::post('levels', [LevelController::class, 'store']);
31  Route::get('levels/{level}', [LevelController::class, 'show']);
32  Route::put('levels/{level}', [LevelController::class, 'update']);
33  Route::delete('levels/{level}', [LevelController::class, 'destroy']);
```

4. Jika sudah. Lakukan uji coba API mulai dari fungsi untuk menampilkan data. Gunakan URL: localhost/PWL_POS-main/public/api/levels dan method GET. Jelaskan dan berikan screenshoot hasil percobaan Anda.

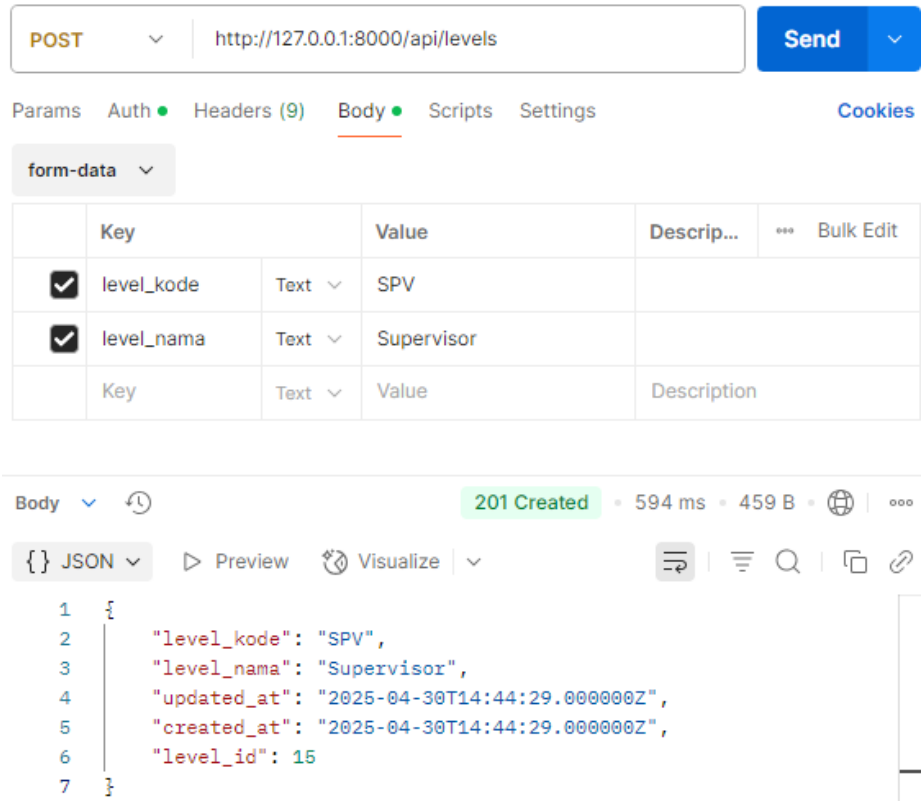




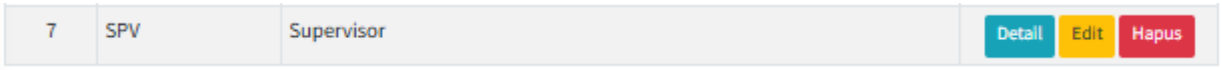
5. Kemudian, lakukan percobaan penambahan data dengan URL : localhost/PWL_POSmain/public/api/levels dan method POST seperti di bawah ini.



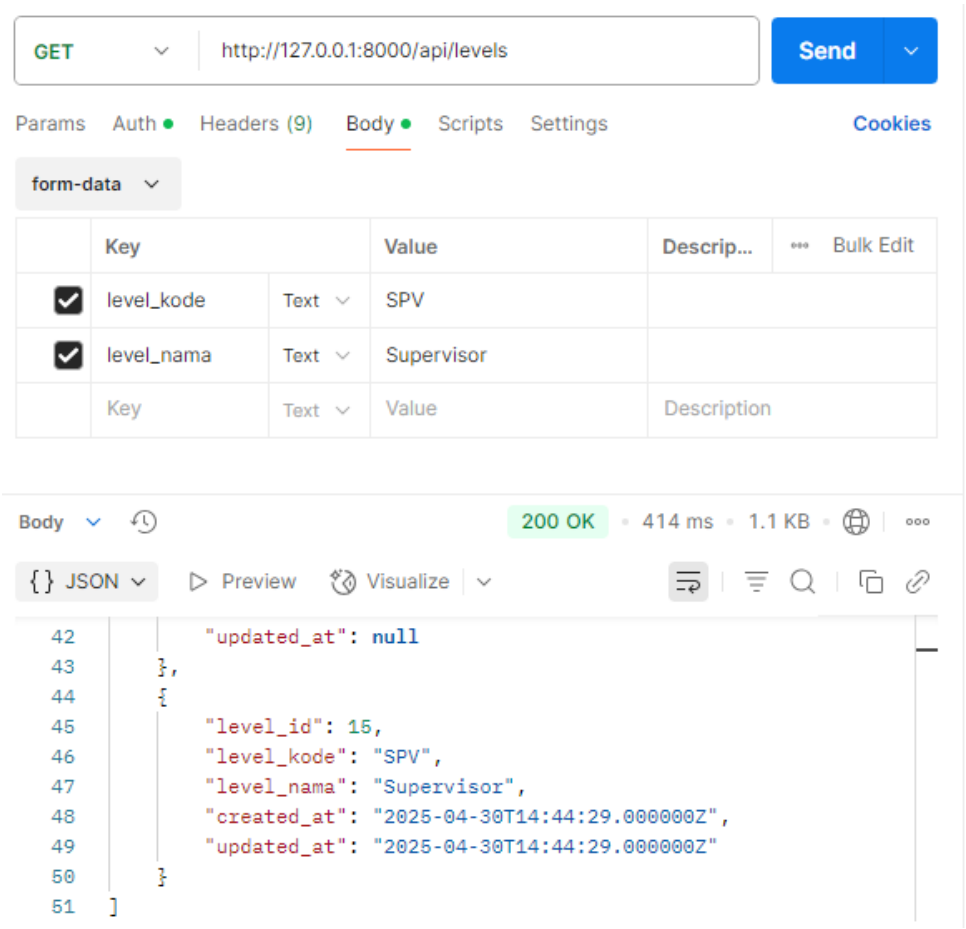
Jelaskan dan berikan screenshoot hasil percobaan Anda.



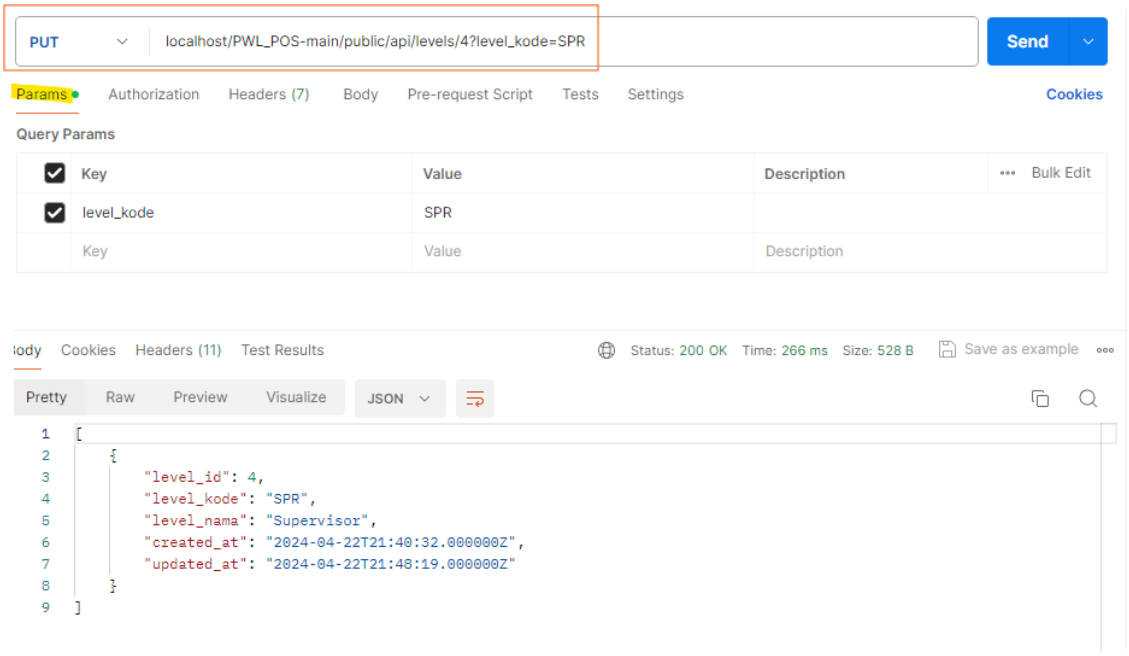
Data Tertambah:



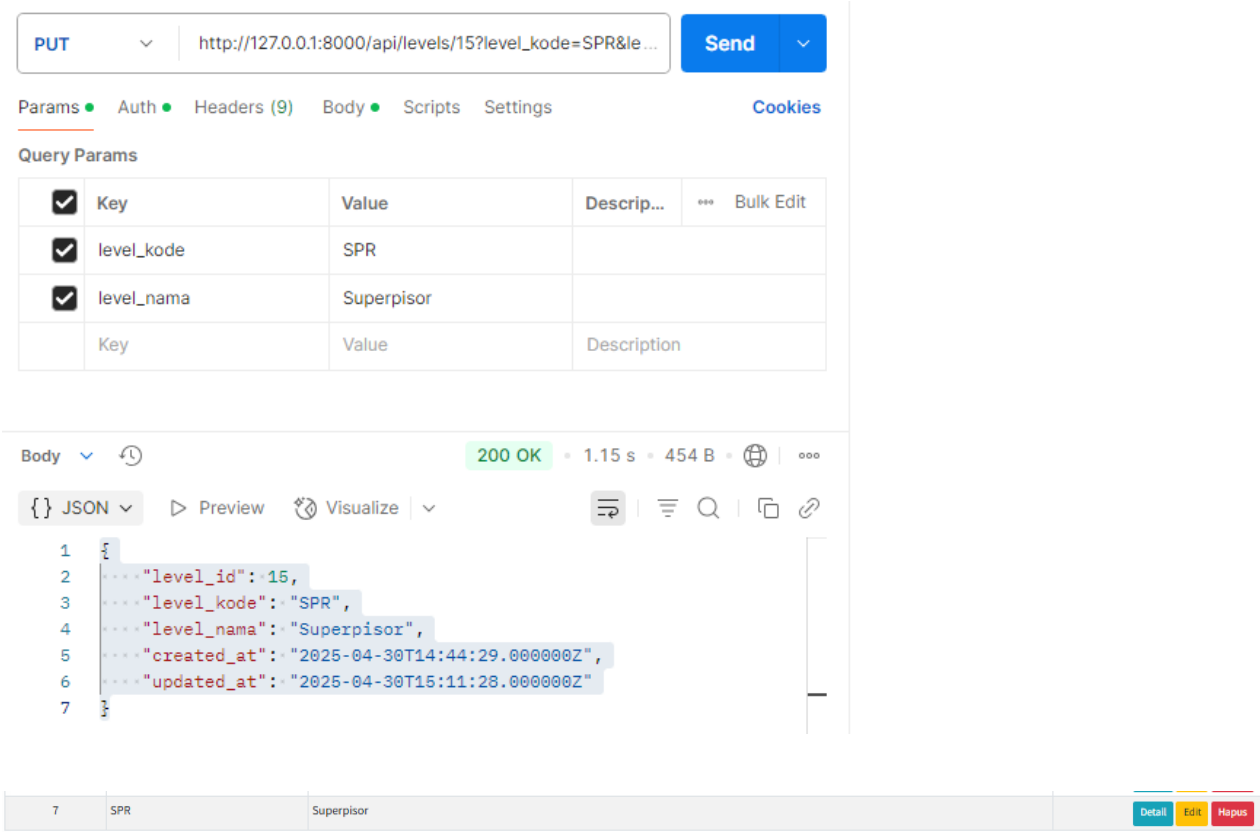
6. Berikutnya lakukan percobaan menampilkan detail data. Jelaskan dan berikan screenshoot hasil percobaan Anda.



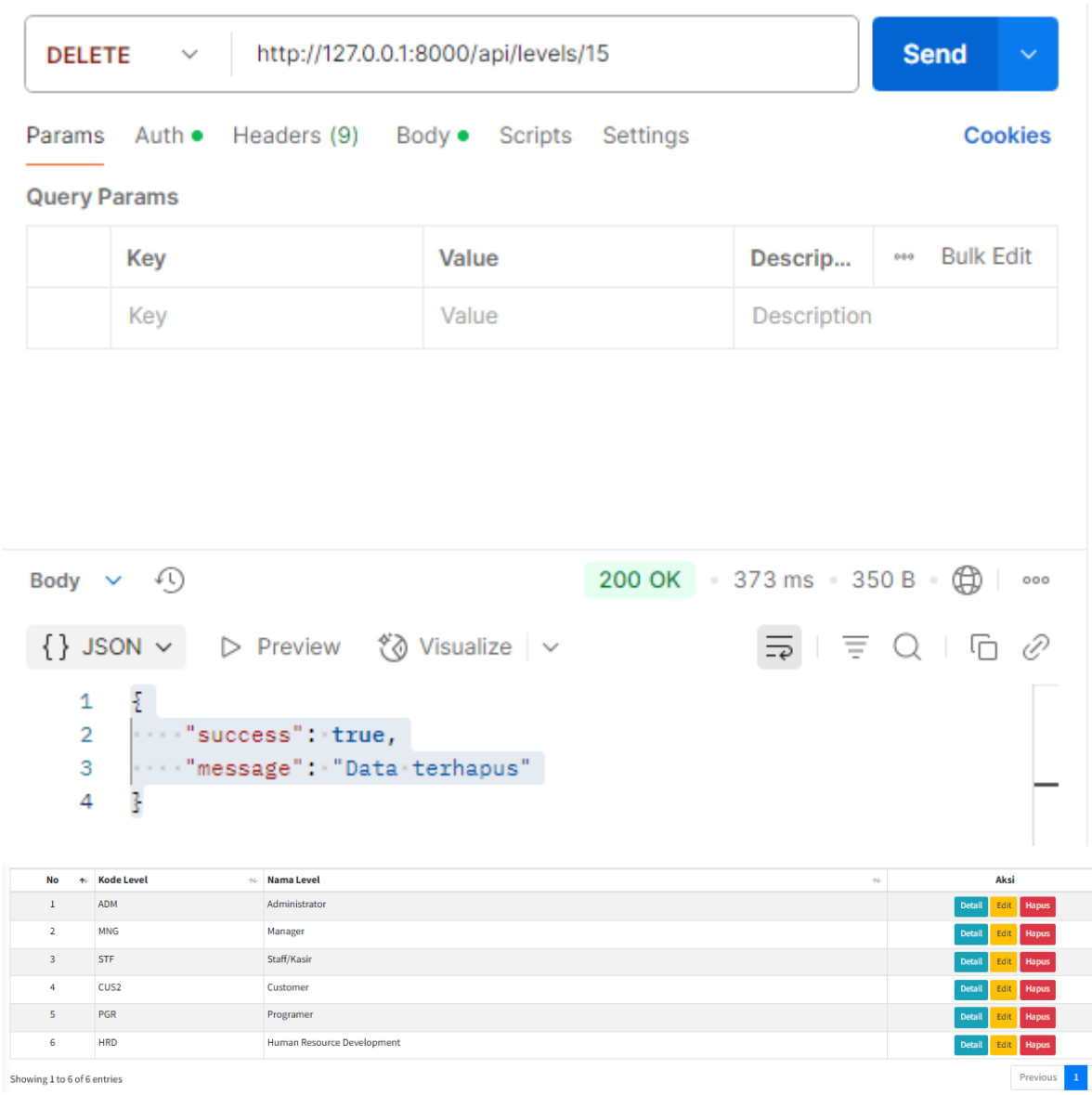
7. Jika sudah, kita coba untuk melakukan edit data menggunakan localhost/PWL_POSmain/public/api/levels/{id} dan method PUT. Isikan data yang ingin diubah pada tab Param.



Jelaskan dan berikan screenshoot hasil percobaan Anda.



8. Terakhir lakukan percobaan hapus data. Jelaskan dan berikan screenshoot hasil percobaan Anda.



9. Lakukan commit perubahan file pada Github.

TUGAS

Implementasikan CRUD API pada tabel lainnya yaitu tabel m_user, m_kategori, dan m_barang

1. **API pada m_user**
   - UserController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/UserControl
ler

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\U
serController.php] created successfully.
```

app > Http > Controllers > Api > UserController.php > PHP > UserController

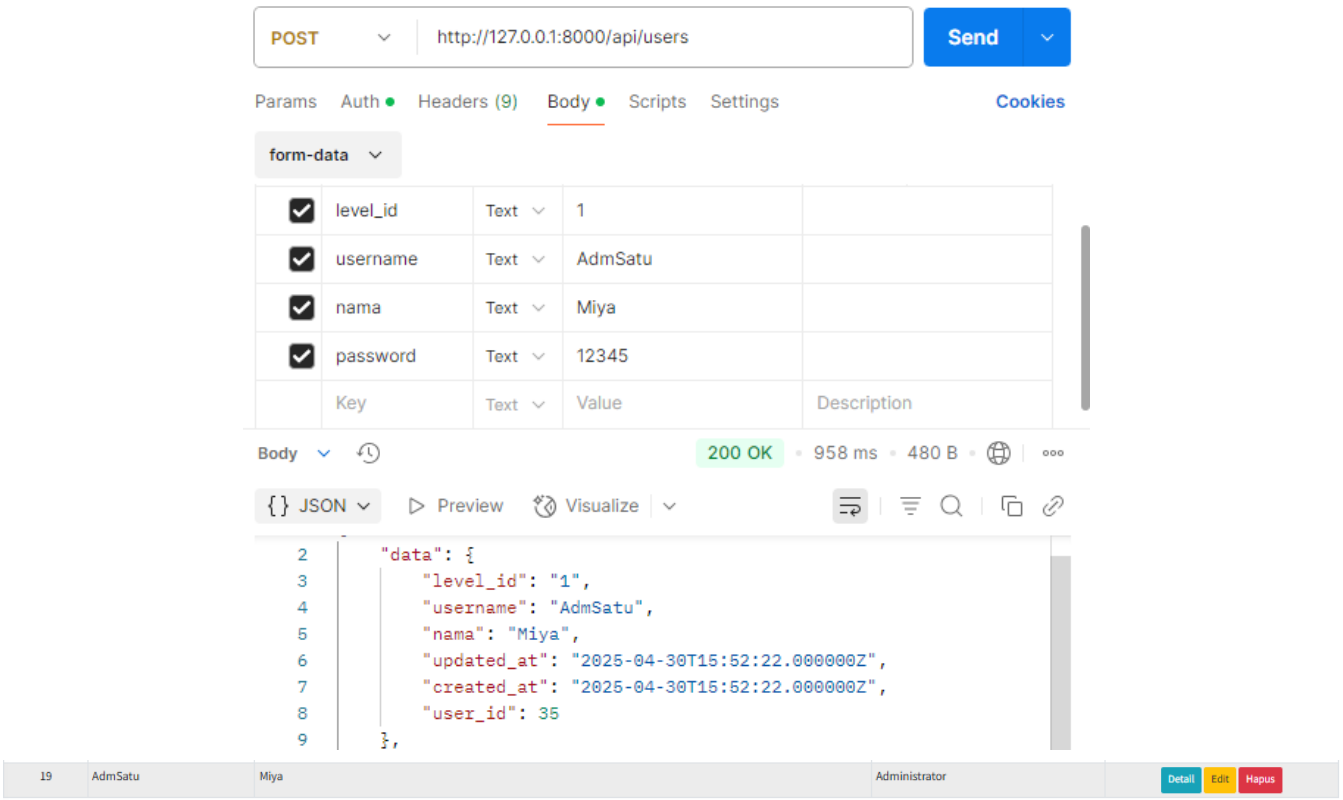
```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\UserModel;
use Illuminate\Http\JsonResponse;

1 reference | 0 implementations
class UserController extends Controller
{
```

```php
    0 references | 0 overrides
    public function index(): Collection
    {
        return UserModel::all();
    }

    0 references | 0 overrides
    public function store(Request $request): JsonResponse|mixed
    {
        $user = UserModel::create($request->all());
        return response()->json(["data" => $user, "status" => 201]);
    }

    0 references | 0 overrides
    public function show(UserModel $user): UserModel
    {
        return $user;
    }

    0 references | 0 overrides
    public function update(Request $request, UserModel $user): UserModel
    {
        $user->update($request->all());
        return $user;
    }

    0 references | 0 overrides
    public function destroy(UserModel $user): JsonResponse|mixed
    {
        $user->delete();
        return response()->json([
            "success" => true,
            "message" => "Data terhapus"
        ]);
    }
}
```

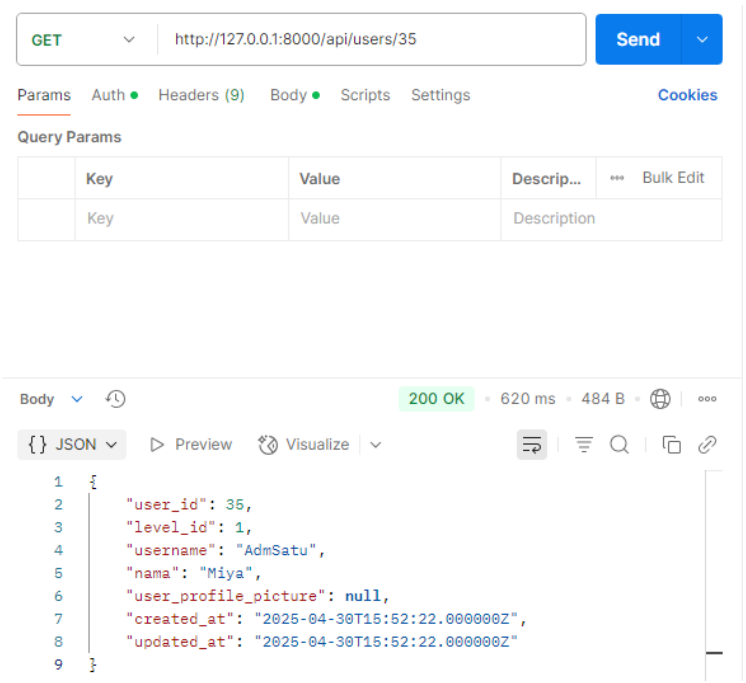
- Route

```php
// API User
Route::get('/users', [UserController::class, 'index']);
Route::post('/users', [UserController::class, 'store']);
Route::get('/users/{user}', [UserController::class, 'show']);
Route::put('/users/{user}', [UserController::class, 'update']);
Route::delete('/users/{user}', [UserController::class, 'destroy']);
```

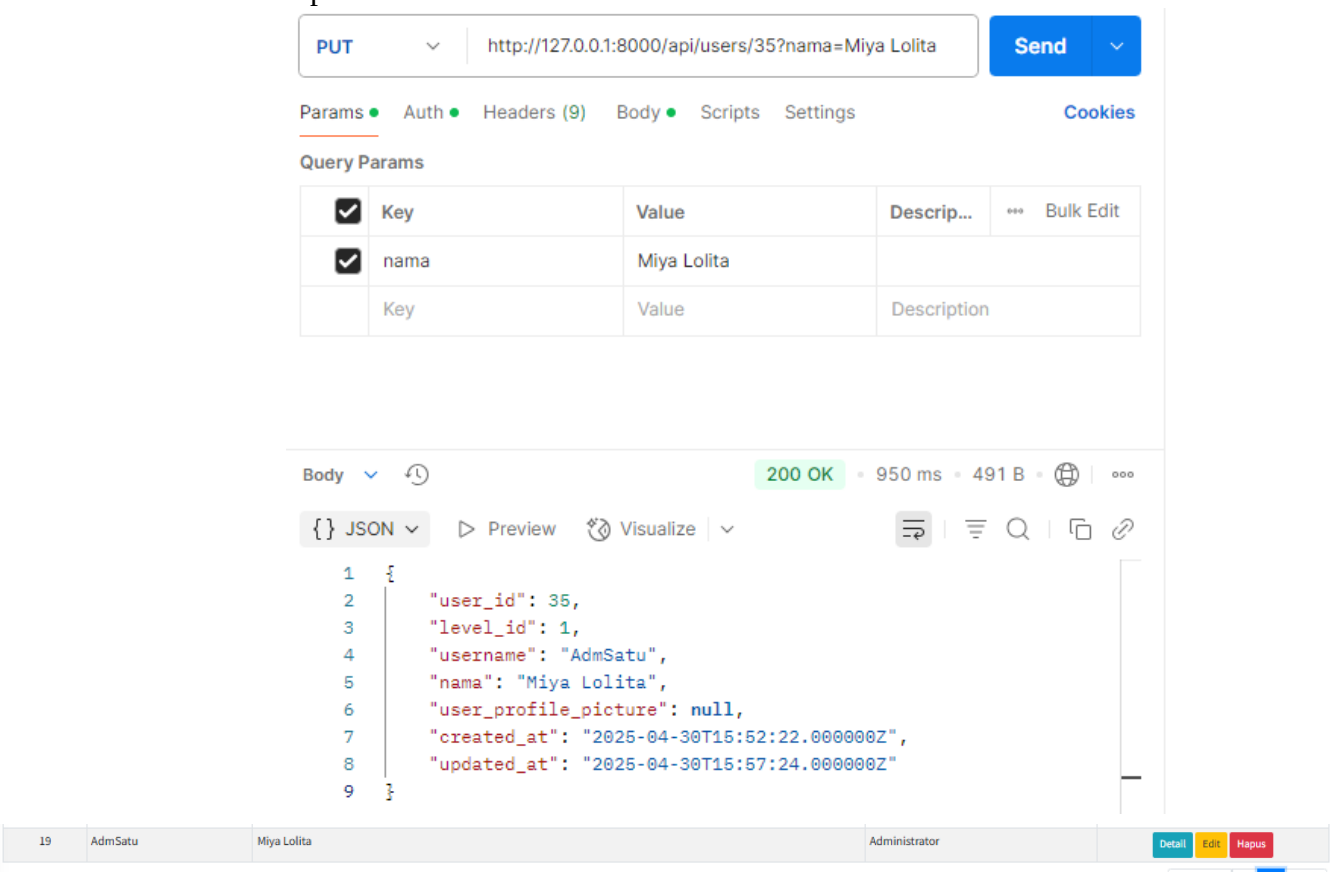
- CRUD
    - create


POST
http://127.0.0.1:8000/api/users
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
form-data
level_id
Text
1
username
AdmSatu
nama
Miya
password
12345
Key
Value
Description
Body
200 OK
958 ms
480 B
JSON
Preview
Visualize
"data": {
"level_id": "1",
"username": "AdmSatu",
"nama": "Miya",
"updated_at": "2025-04-30T15:52:22.000000Z",
"created_at": "2025-04-30T15:52:22.000000Z",
"user_id": 35
},
19
AdmSatu
Miya
Administrator
Detail
Edit
Hapus

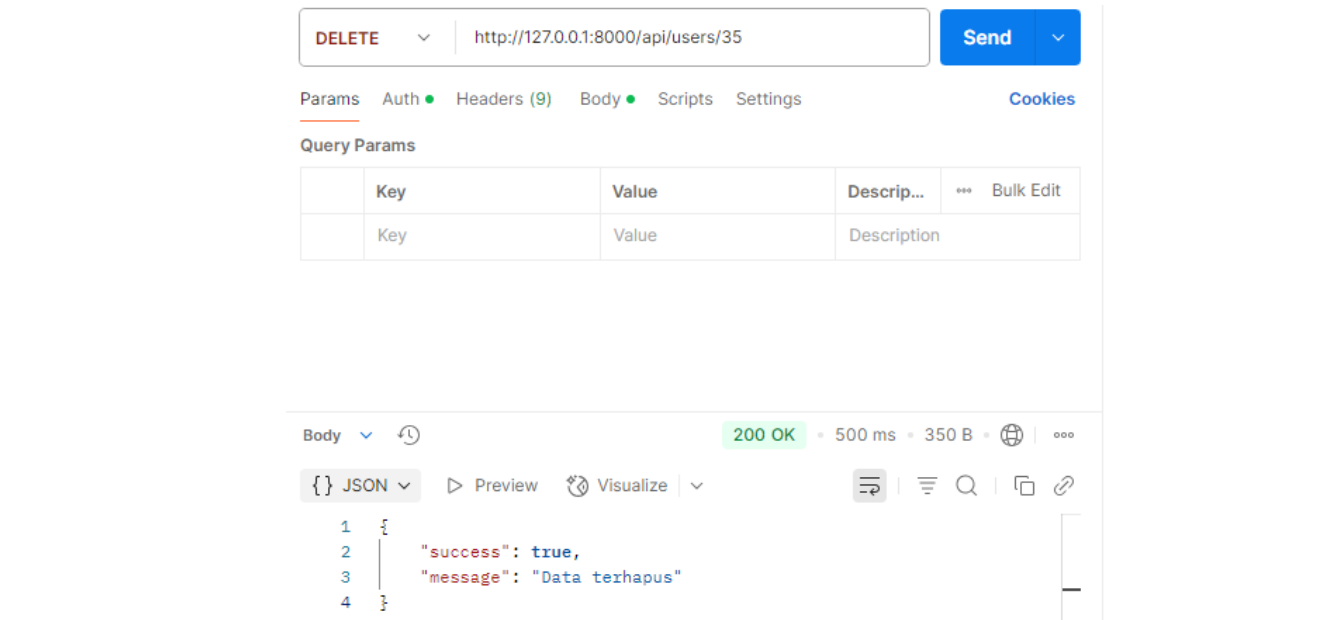
- Read


GET
http://127.0.0.1:8000/api/users/35
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Descrip...
Bulk Edit
Key
Value
Description
Body
200 OK
620 ms
484 B
JSON
Preview
Visualize
{
"user_id": 35,
"level_id": 1,
"username": "AdmSatu",
"nama": "Miya",
"user_profile_picture": null,
"created_at": "2025-04-30T15:52:22.000000Z",
"updated_at": "2025-04-30T15:52:22.000000Z"
}

- Update



- Delete



2. API pada m_kategori

- KategoriController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/KategoriCont
roller

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\Ka
tegoriController.php] created successfully.
```

app > Http > Controllers > Api > KategoriController.php > PHP > KategoriController

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\KategoriModel;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Http\JsonResponse;

1 reference | 0 implementations
class KategoriController extends Controller
{
```

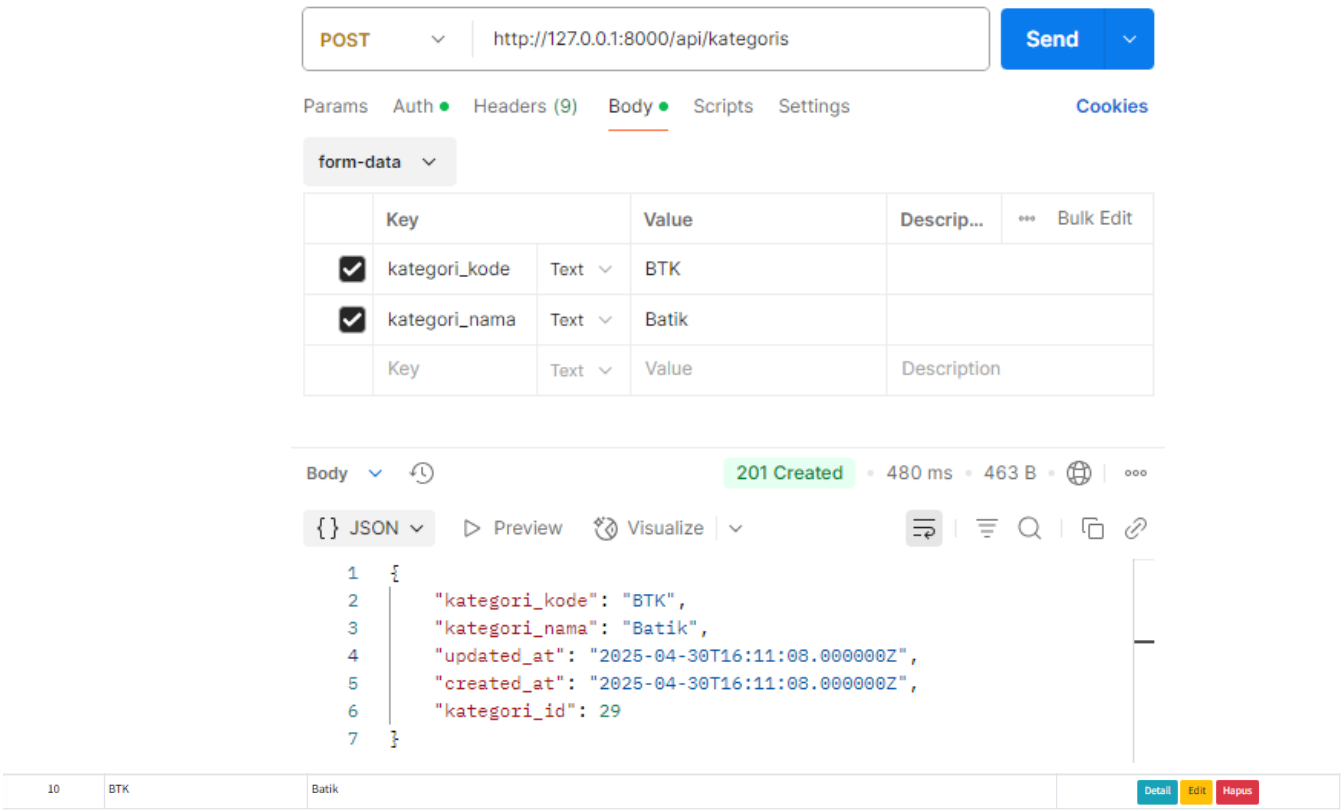
```php
    public function index(): Collection
    {
        return KategoriModel::all();
    }

    0 references | 0 overrides
    public function store(Request $request): JsonResponse|mixed
    {
        $kategori = KategoriModel::create($request->all());
        return response()->json($kategori, 201);
    }

    0 references | 0 overrides
    public function show(KategoriModel $kategori): KategoriModel
    {
        return $kategori;
    }

    0 references | 0 overrides
    public function update(Request $request, KategoriModel $kategori): KategoriModel
    {
        $kategori->update($request->all());
        return $kategori;
    }

    0 references | 0 overrides
    public function destroy(KategoriModel $kategori): JsonResponse|mixed
    {
        $kategori->delete();

        return response()->json([
            'success' => true,
            'message' => 'Data kategori terhapus',
        ]);
    }
}
```

- Routes

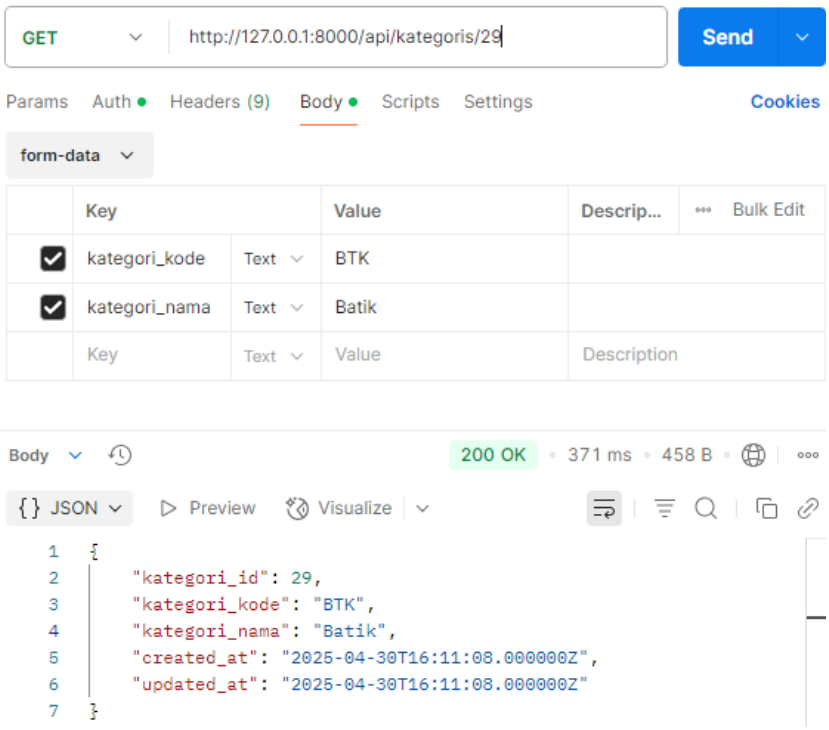
```php
use App\Http\Controllers\Api\KategoriController;
```

```php
// API Kategori
Route::get('/kategoris', [KategoriController::class, 'index']);
Route::post('/kategoris', [KategoriController::class, 'store']);
Route::get('/kategoris/{kategori}', [KategoriController::class, 'show']);
Route::put('/kategoris/{kategori}', [KategoriController::class, 'update']);
Route::delete('/kategoris/{kategori}', [KategoriController::class, 'destroy']);
```

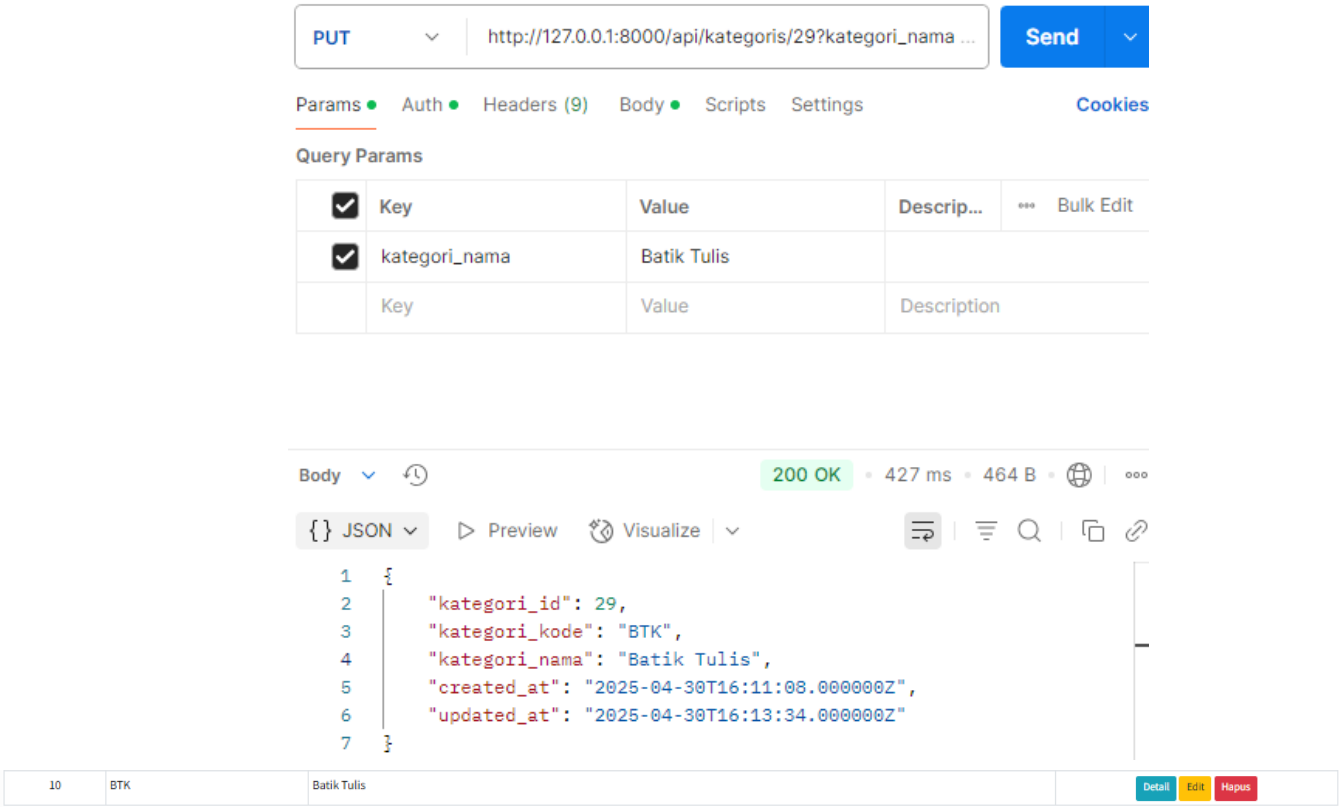
- CRUD
    - Create


POST
http://127.0.0.1:8000/api/kategoris
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
form-data
Key
Value
Descrip...
Bulk Edit
kategori_kode
Text
BTK
kategori_nama
Text
Batik
Key
Text
Value
Description
Body
201 Created
480 ms
463 B
JSON
Preview
Visualize
{
"kategori_kode": "BTK",
"kategori_nama": "Batik",
"updated_at": "2025-04-30T16:11:08.000000Z",
"created_at": "2025-04-30T16:11:08.000000Z",
"kategori_id": 29
}
10
BTK
Batik
Detail
Edit
Hapus

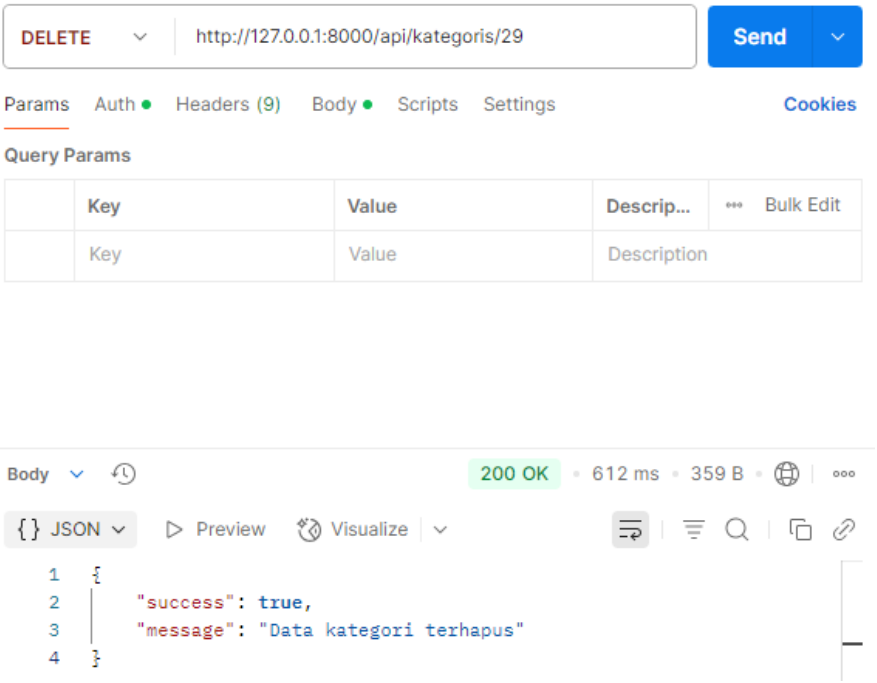
- Read


GET
http://127.0.0.1:8000/api/kategoris/29
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
form-data
Key
Value
Descrip...
Bulk Edit
kategori_kode
Text
BTK
kategori_nama
Text
Batik
Key
Text
Value
Description
Body
200 OK
371 ms
458 B
JSON
Preview
Visualize
{
"kategori_id": 29,
"kategori_kode": "BTK",
"kategori_nama": "Batik",
"created_at": "2025-04-30T16:11:08.000000Z",
"updated_at": "2025-04-30T16:11:08.000000Z"
}

- Update



- Delete



3. API pada m_barang

- BarangController

```
PS C:\laragon\www\PWL2025\Week10\Jobsheet10> php artisan make:controller Api/BarangController

  INFO  Controller [C:\laragon\www\PWL2025\Week10\Jobsheet10\app\Http\Controllers\Api\BarangCon
troller.php] created successfully.
```

app > Http > Controllers > Api > BarangController.php > PHP > BarangController

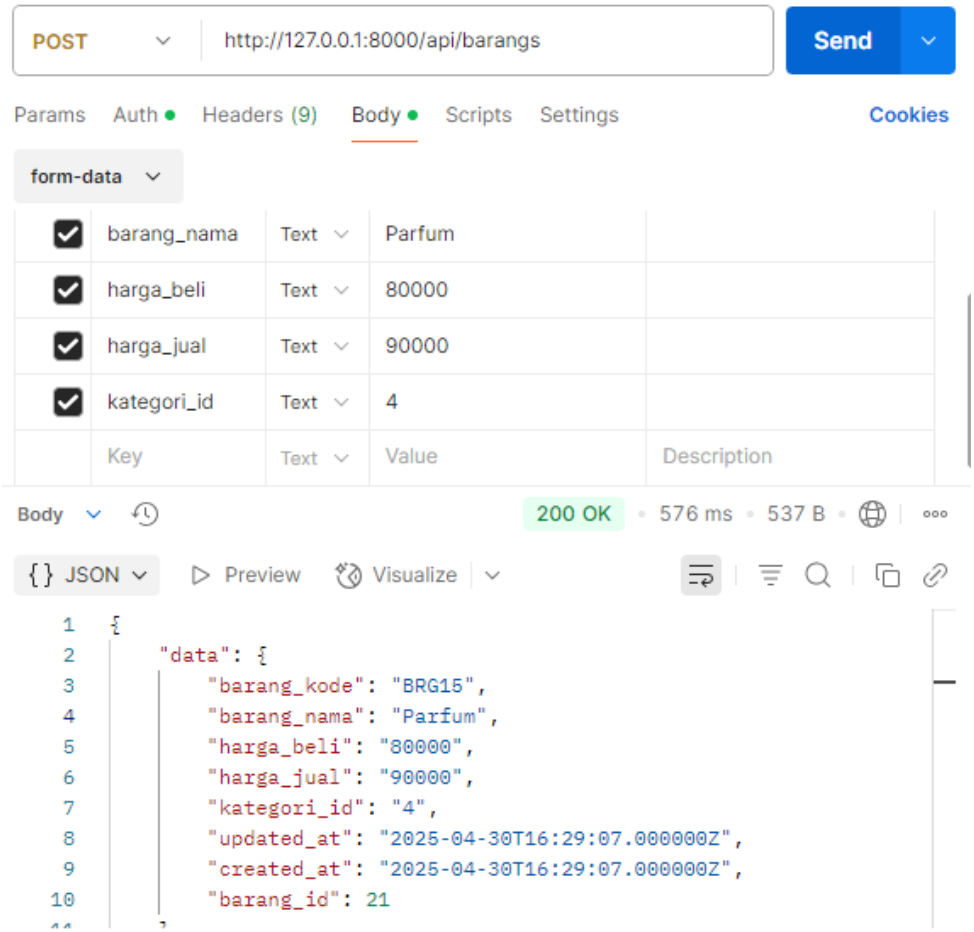
```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\BarangModel;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Http\JsonResponse;

1 reference | 0 implementations
class BarangController extends Controller
{
```

```php
    public function index(): Collection
    {
        return BarangModel::all();
    }

    0 references | 0 overrides
    public function store(Request $request): JsonResponse|mixed
    {
        $barang = BarangModel::create($request->all());
        return response()->json([
            'data' => $barang,
            'status' => 201
        ]);
    }

    0 references | 0 overrides
    public function show(BarangModel $barang): BarangModel
    {
        return $barang;
    }

    0 references | 0 overrides
    public function update(Request $request, BarangModel $barang): BarangModel
    {
        $barang->update($request->all());
        return $barang;
    }

    0 references | 0 overrides
    public function destroy(BarangModel $barang): JsonResponse|mixed
    {
        $barang->delete();
        return response()->json([
            'message' => 'Data terhapus.'
        ], 200);
    }
}
```

- Routes

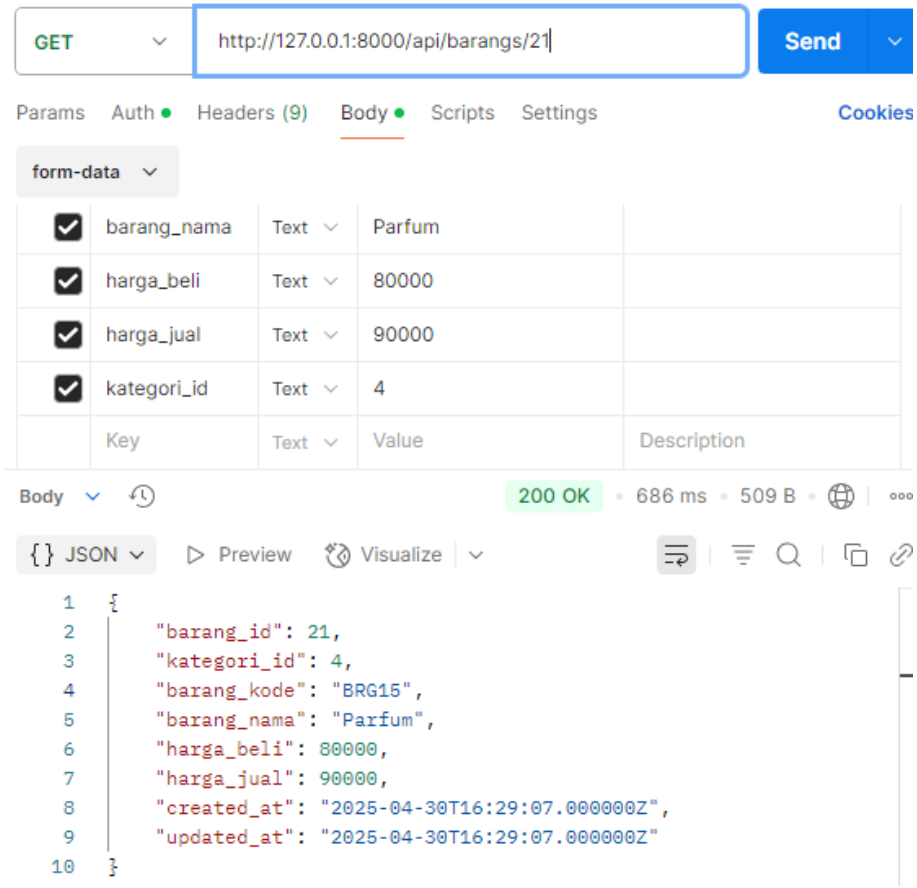
```php
use App\Http\Controllers\Api\BarangController;

// API Barang
Route::get('/barangs', [BarangController::class, 'index']);
Route::post('/barangs', [BarangController::class, 'store']);
Route::get('/barangs/{barang}', [BarangController::class, 'show']);
Route::put('/barangs/{barang}', [BarangController::class, 'update']);
Route::delete('/barangs/{barang}', [BarangController::class, 'destroy']);
```

- CRUD
    - Create

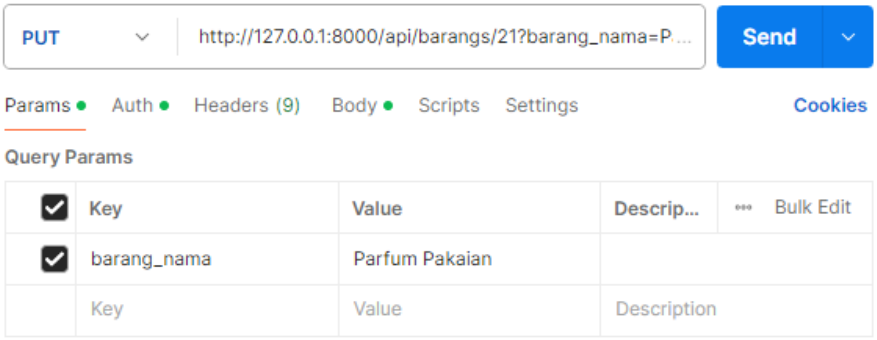

POST
http://127.0.0.1:8000/api/barangs
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
form-data
barang_nama
Text
Parfum
harga_beli
Text
80000
harga_jual
Text
90000
kategori_id
Text
4
Key
Text
Value
Description
Body
200 OK
576 ms
537 B
JSON
Preview
Visualize
{
"data": {
"barang_kode": "BRG15",
"barang_nama": "Parfum",
"harga_beli": "80000",
"harga_jual": "90000",
"kategori_id": "4",
"updated_at": "2025-04-30T16:29:07.000000Z",
"created_at": "2025-04-30T16:29:07.000000Z",
"barang_id": 21

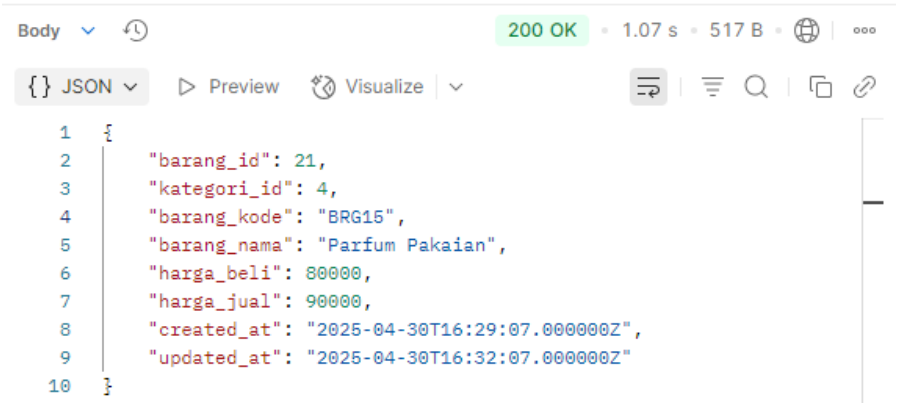

17
4
BRG15
Parfum
80.000
90.000
Pewangi
Detail
Edit
Hapus

- Read


GET
http://127.0.0.1:8000/api/barangs/21
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
form-data
barang_nama
Text
Parfum
harga_beli
Text
80000
harga_jual
Text
90000
kategori_id
Text
4
Key
Text
Value
Description
Body
200 OK
686 ms
509 B
{} JSON
Preview
Visualize
{
"barang_id": 21,
"kategori_id": 4,
"barang_kode": "BRG15",
"barang_nama": "Parfum",
"harga_beli": 80000,
"harga_jual": 90000,
"created_at": "2025-04-30T16:29:07.000000Z",
"updated_at": "2025-04-30T16:29:07.000000Z"
}

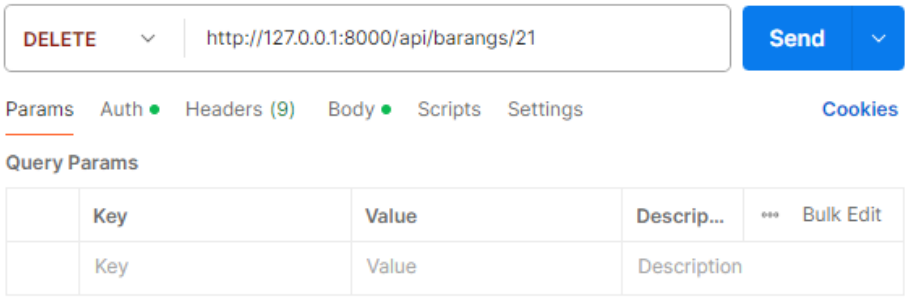
- Update


PUT
http://127.0.0.1:8000/api/barangs/21?barang_nama=P...
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Descrip...
Bulk Edit
barang_nama
Parfum Pakaian
Key
Value
Description


Body
200 OK
1.07 s
517 B
JSON
Preview
Visualize
{
"barang_id": 21,
"kategori_id": 4,
"barang_kode": "BRG15",
"barang_nama": "Parfum Pakaian",
"harga_beli": 80000,
"harga_jual": 90000,
"created_at": "2025-04-30T16:29:07.000000Z",
"updated_at": "2025-04-30T16:32:07.000000Z"
}


17
4
BRG15
Parfum Pakaian
80.000
90.000
Pewangi
Detail
Edit
Hapus

- Delete


DELETE
http://127.0.0.1:8000/api/barangs/21
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Descrip...
Bulk Edit
Key
Value
Description


No
ID Kategori
Kode Barang
Nama Barang
Harga Beli
Harga Jual
Kategori
Aksi
11 2 BRG011 lip cream 15.000 18.000 MakeUp
12 1 SBK-003 Telur Omega(10 butir) 22.000 25.000 Makanan
13 2 SNK-003 Sari Roti 11.500 12.500 MakeUp
14 3 MND-003 Shampo Pantene 17.500 18.500 Aksesoris
15 4 BAY-003 Baju Bayi 2th 89.000 92.500 Pewangi
16 5 MNM-003 Cleo 600ml 3.750 4.300 Alat Mandi
Detail
Edit
Hapus
Showing 11 to 16 of 16 entries
Previous
1
2
Next

**** Sekian, dan selamat belajar ****